



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE**

TITLUL LUCRĂRII DE DISERTAȚIE

LUCRARE DE DISERTAȚIE

Absolvent masterand: **Prenume NUME**

Coordonator științific: **titlul științific Prenume NUME**

Iulie, 2026



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN
Prof.Dr.Ing. Vlad MUREȘAN

DIRECTOR DEPARTAMENT
Prof.Dr.Ing. Rodica POTOLEA

Student masterand: **Prenume NUME**

TITLUL LUCRĂRII DE DISERTAȚIE

1. **Enunțul temei:** *Scurtă descriere a temei lucrării de disertație și datele inițiale*
2. **Conținutul lucrării:** *(enumerarea părților componente) Exemplu: Pagina de prezentare, aprecierile coordonatorului, titlul capitolului 1, titlul capitolului 2, ..., titlul capitolului n, bibliografie, anexe.*
3. **Locul documentării:** *Exemplu: UTCN, Cluj-Napoca*
4. **Consultanți:**
5. **Data emiterii temei:** *Exemplu: noiembrie 2025*
6. **Data predării:** *Exemplu: 5 iulie 2026*

Student masterand:

Coordonator științific:

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE**
DEPARTAMENTUL CALCULATOARE**Declarație pe proprie răspundere privind
autenticitatea lucrării de disertație**

Subsemnatul(a) Prenume NUME, legitimat(ă) cuserianr., CNP, autorul lucrării TITLUL LUCRĂRII DE DISERTAȚIE, elaborată în vederea susținerii examenului de finalizare a studiilor de disertație la Facultatea de Automatică și Calculatoare, Programul de studii, din cadrul Universității Tehnice din Cluj-Napoca, sesiunea (iulie/septembrie) a anului universitar 2025-2026, declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de disertație*.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine.

Data

.....

Prenume NUME

.....

Semnătura

Cuprins

Capitolul 1	Introducere	1
Capitolul 2	Obiectivele cercetării	6
Capitolul 3	Studiu bibliografic/Stadiul actual în domeniu	9
3.1	Distcc	9
3.2	Ccache	11
3.3	Sccache	13
3.4	DiSCo	13
Capitolul 4	Prezentarea proiectului	15
4.1	Fundamentare teoretică	15
4.1.1	Sisteme distribuite	15
4.1.2	Mecanisme de concurență și paralelism	17
4.1.3	Funcționarea procesului de compilare pentru C/C++	22
4.1.4	Sisteme de build pentru C/C++	24
4.2	Arhitectura conceptuală a sistemului	28
4.2.1	Interceptorul pentru comenzile de compilare	30
4.2.2	Aplicația client	33
4.2.3	Aplicația server	38
4.3	Aspecte de implementare	41
4.3.1	Implementarea protocolului de comunicație	41
4.3.2	Asigurarea concurenței în procesare și comunicare	46
Capitolul 5	Rezultate teoretice și experimentale	50
Capitolul 6	Concluzii	51
6.1	Conținut	51
6.2	Detalii tehnice	51
6.2.1	Dimensiune	51
	Bibliography	52

Capitolul 1

Introducere

Limbaajele de programare compilate, precum **C** și **C++**, sunt de cele mai multe ori alegerea corectă pentru dezvoltarea de sisteme și aplicații pentru care performanța în timpul rulării este un aspect de importanță critică, însă alegerea lor prezintă inconveniente în ceea ce privește procesul de build. Cerințele ridicate de memorie și timpii îndelungați de compilare cresc proporțional cu dimensiunea proiectului aflat în dezvoltare.

Timpii îndelungați de compilare sunt de natură să încetinească drastic procesul de dezvoltare, ceea ce determină întârzieri ce afectează livrarea produsului către clienți și pierderi costisitoare de timp pentru evoluția proiectului.

Evident, investiția în hardware mai performant pentru a reduce timpul de compilare poate diminua magnitudinea problemei, însă se estimează că în următorii zece ani, puterea de procesare se va satura (legea lui Moor nu va mai fi valabilă începând din 2036 [1]), iar accentul se va muta asupra optimizării resurselor de rețea. Progresele în domeniul comunicațiilor vor permite transferuri rapide și eficiente de date între nodurile sistemelor distribuite, susținând astfel implementarea și extinderea proceselor distribuite de compilare. Această abordare permite fragmentarea și procesarea paralelă a sarcinilor de build pe mai multe noduri din rețea, fie ele stații locale, servere dedicate sau resurse din cloud. Drept rezultat, proiectele software vor putea beneficia de timpi reduși de compilare și de o utilizare mai eficientă a resurselor disponibile, contribuind la creșterea productivității și la reducerea blocajelor din ciclurile de dezvoltare.

Compilarea folosind mai multe sisteme de calcul care comunică prin rețea este susținută și de faptul că, odată cu răspândirea utilizării generale a calculatoarelor și a resurselor cloud, timpii de neutilizare ai sistemelor (Idle Times), au crescut considerabil. În instituții educaționale, calculatoarele au o durată de neutilizare medie de 28% (adică 2672 de ore din total de 9550 de ore de funcționare) [2]. Așadar, exploatarea resurselor de calcul ale sistemelor neutilizate reprezintă o eficientizare binevenită pentru reducerea duratei de build.

Pentru a putea înțelege de ce durata procesului de compilare pentru **C/C++** (dar mai ales pentru **C++**) este semnificativ mai mare față de alte limbaaje de programare, găsim esențial să explicăm modul de desfășurare al acestui proces.

Transformarea codului sursă într-un fișier executabil este compusă din trei etape majore. Primul pas îl reprezintă expandarea directivelor de preprocesare, moment în care dependențele dintre fișierele sursă ale proiectului sunt determinate. Rezultatul etapei de preprocesare îl reprezintă o colecție de unități de translație (Translation Units), fiecare fișier sursă având asociat un astfel de **translation unit**. Urmează etapa de compilare, în urma căreia unitățile de translație sunt transformate în fișiere ce conțin cod mașină, numite fișiere obiect (Object Files). În ultima etapa, numită link-editare, toate fișierele obiect rezultate în urma compilării sunt conectate pentru a genera fișierul executabil al proiectului [3].

Etapa de preprocesare este [3] responsabilă de pregătirea fișierelor sursă pentru a fi furnizate ca intrare pentru compilator. În **C/C++**, fiecare simbol poate fi definit o singură dată, însă declararea simbolului poate să apară de mai multe ori, întrucât fiecare simbol utilizat într-o unitate de translație, trebuie să aibă o declarație locală. Această chestiune este tratată de mecanismul de includere a fișierelor, folosind directiva de preprocesare **#include**. Figura 1.1 exemplifică mecanismul de preprocesare al fișierelor sursă.

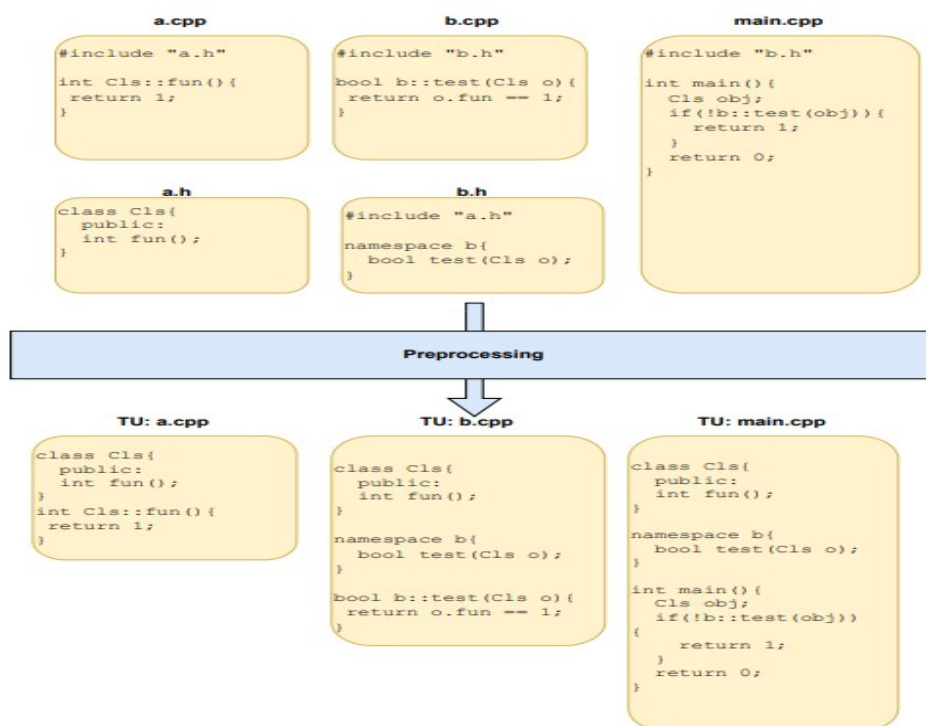


Figura 1.1: Preprocesarea fișierelor sursă [3]

Unitățile de translație sunt furnizate ca date de intrare pentru compilator, care detectează erorile de sintaxă și produce fișierele obiect, care deși conțin cod mașină, nu pot fi încă executate. Figura exemplifică etapa de compilare.

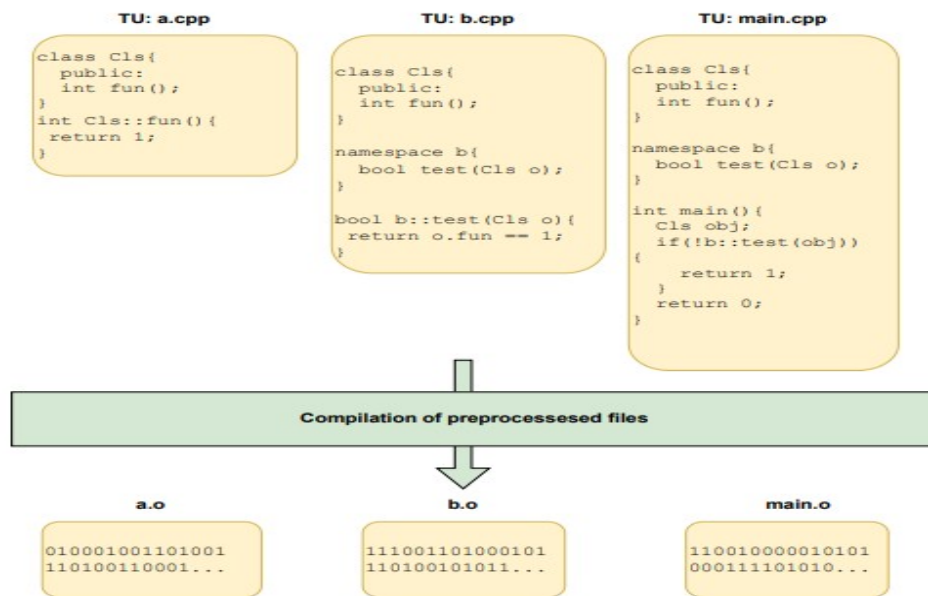


Figura 1.2: Compilarea unităților de translație [3]

Etapa de linking [3] conectează fișierele obiect pentru ca declarațiile locale să fie disponibile în celelalte unități de translație. Ieșirea linker-ului reprezintă un fișier executabil, care este rezultatul final al procesului de build.

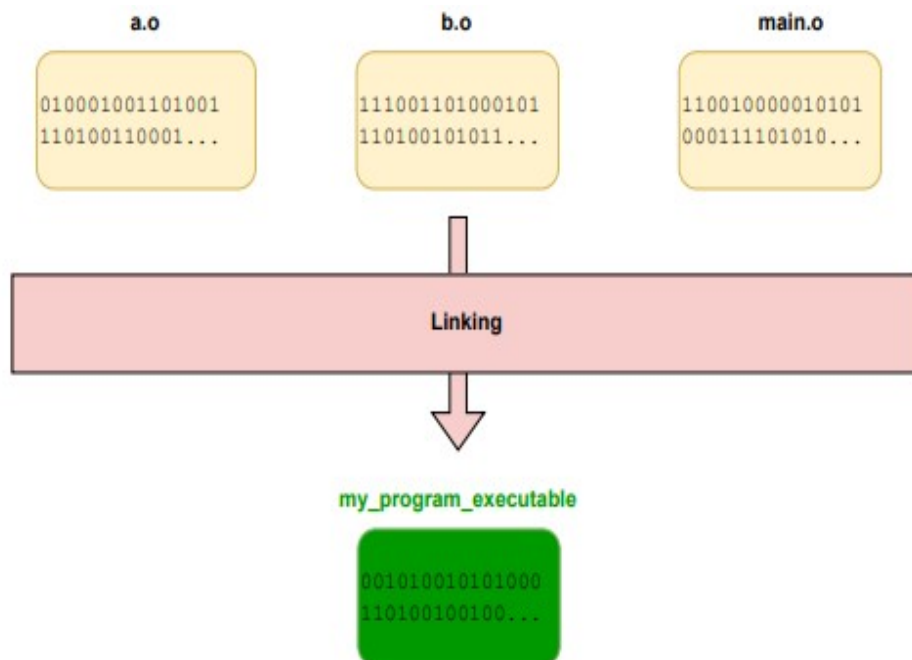


Figura 1.3: Generearea fișierului executabil [3]

Proiectele complexe, în care numărul de dependențe este mare, suferă des de recompilarea unui număr mare de fișiere, ceea ce crește semnificativ durata de compilare. Motivul pentru care compilarea excesivă este greu de evitat îl reprezintă faptul că, dacă un fișier header care este inclus de un număr mare de fișiere sursă este modificat, atunci se declanșează recompilarea tuturor fișierelor sursă care depind de acel header. Când numărul de fișiere header incluse este mare, există șansa să se creeze cicluri de dependență, o situație în care mai multe fișiere se includ reciproc [4].

Totodată, mecanismul de **templating** din **C++**, permite definirea de clase și funcții generice, care pot fi instanțiate folosind tipuri de date diverse. Întrucât **C++** este un limbaj de programare cu sistem static de tipuri (statically typed), de fiecare data când un template este invocat pentru un nou tip de argument, codul pentru implementarea specifică acelui tip de date este generat în mod automat de către compilator [5]. În cazul în care template-ul este instanțiat pentru foarte multe tipuri de date diferite, compiler-ul generează un volum semnificativ de cod repetitiv, ceea ce poate încetini procesul de build [6].

Pentru a atrage atenția asupra scenariilor în care timpul de build devine o problemă, voi invoca creșterea exponențială a volumului de cod sursă care intră în componența unor proiecte open-source cunoscute, precum **Linux Kernel** și **Android**[7]. Dimensiunea codebase-ului pentru acestea se ridică la ordinul zecilor de milioane de linii de cod. Creșterea volumului de cod de-a lungul timpului este vizibilă în Figura 1.4.

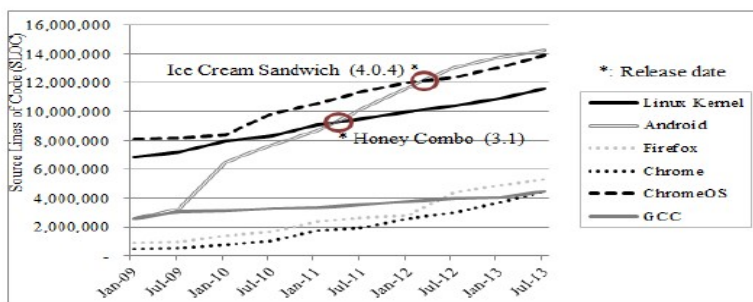


Figura 1.4: Creșterea codebase-ului de-a lungul anilor pentru câteva proiecte open-source [8]

Pentru sistemul de operare mobil **Android 4.2.2**, timpul de build total ajunge la 51 de minute [8].

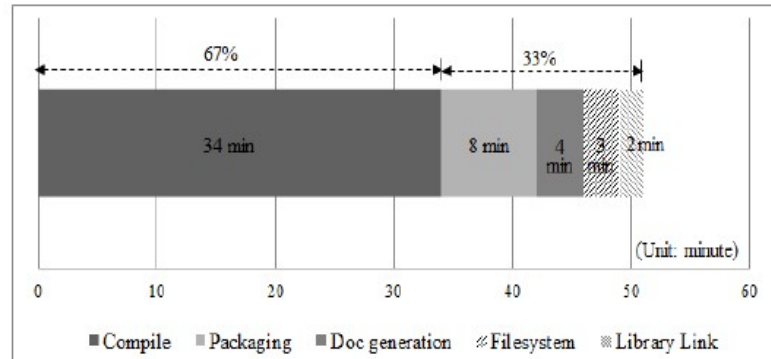


Figura 1.5: Timp de build Android [8]

Capitolul 2

Obiectivele cercetării

Obiectivul principal al cercetării constă în evaluarea tehnologiilor moderne de comunicare din domeniul sistemelor distribuite, în scopul implementării unui nou sistem de compilare distribuită pentru proiecte C/C++, care să ofere performanțe competitive în ceea ce privește eficientizarea procesului de compilare, comparativ cu alternativele deja existente. Sistemul dezvoltat propune o abordare eficientă, scalabilă și flexibilă din punct de vedere arhitectural, de a crește viteza procesului de compilare, fără a introduce incompatibilități cu uneltele deja folosite în dezvoltare proiectelor.

Compilarea proiectelor dezvoltate folosind C/C++ este compusă din etape multiple de transformare a codului sursă: preprocesarea, compilarea, generarea de cod mașină pentru platforma vizată și link-editarea. Unele operațiuni aferente procesului de compilare pot fi executate independent, în paralel, însă gestiunea dependențelor dintre fișierele componente ale proiectului, artefactele generate și setările de configurare ale compilatorului introduce un grad însemnat de complexitate. Un obiectiv ce trebuie îndeplinit de către orice sistem distribuit de acest tip îl reprezintă identificarea corectă a sarcinilor care pot fi preluate de către sistemele de calcul auxiliare, spre a fi paralelizate, astfel încât să fie menținută corectitudinea și capacitatea de reproducere facilă a procesului de compilare.

Lucrarea investighează modul în care sarcinile de compilare pot să fie distribuite într-o manieră eficientă către mai multe noduri de calcul din rețea, pentru a se minimiza suprautilizarea resurselor de calcul, precum și a canalelor de comunicare. De asemenea, sistemul trebuie să păstreze în evidență disponibilitatea sistemelor ce contribuie la procesul de build și să își mențină o stare de funcționare consistentă în cazul apariției erorilor în timpul transmisiei de date în rețea.

Astfel, metodologia de întocmire a lucrării de cercetare propune realizarea unei serii de mai multe activități, pentru a explora modul de funcționare și utilitatea conceptului de compilare distribuită pentru proiecte dezvoltate folosind limbajele de programare C și C++:

- Studiul bibliografic - în vederea dezvoltării unui sistem de compilare distribuită, este necesară o analiză aprofundată a soluțiilor deja existente pentru realizarea acestui tip de sarcină. Studiul bibliografic oferă o imagine de ansamblu a principiilor ce stau la baza compilării distribuite, a implicațiilor practice ale aplicării unei astfel de abordări, precum și a cazurilor de utilizare cele mai frecvente pentru astfel de sisteme
- Proiectarea arhitecturii sistemului - aceasta reprezintă o etapă premergătoare implementării concrete și cuprinde descrierea comportamentului pentru modulele care intră în componența sistemului, cu modalitățile de funcționare și interacțiune aferente. De asemenea, se studiază posibilitatea aplicării unor concepte moderne pentru dezvoltarea sistemelor distribuite și se definește structura arhitecturii hardware și software a ansamblului
- Implementarea sistemului - pe baza arhitecturii definite în etapa anterioară, se implementează soluții practice pentru îndeplinirea operațiunilor asociate funcționării sistemului de compilare distribuită: prelucrarea datelor de intrare, balansarea sarcinii de compilare între nodurile participante, stocarea artefactelor rezultate, gestiunea comunicării dintre noduri pe baza unui protocol predefinit
- Crearea unui mediu de test - validarea funcționării corecte a sistemului dezvoltat poate avea loc doar în contextul utilizării unei configurații corespunzătoare, care să fie cât mai apropiată de condițiile de operare reale, în ceea ce privește dispozitivele hardware implicate, atât din punct de vedere al puterii de calcul, dar și al resurselor de rețea
- Analiza rezultatelor obținute - pe baza metricilor ce caracterizează performanțele și comportamentul sistemului se pot identifica principalele avantaje și dezavantaje ale abordării alese, raportate la celelalte alternative propuse în literatura de specialitate, dar și scenariile optime de utilizare pentru distribuirea procesului de compilare folosind mai multe sisteme de calcul

Pentru a restrânge aria de cercetare asupra conceptului de compilare distribuită pentru **C/C++**, nu se vor studia amănunțit toate aspectele adiacente funcționării unui astfel de sistem, precum testarea mai multor configurații de rețea, asigurarea funcționării compatibilității aplicației cu platform de rulare multiple sau optimizarea procesului de deployment.

Se urmărește dezvoltarea unui sistem distribuit care să fie compatibil cu sistemele de operare bazate pe **Linux** și uneltele software uzuale în dezvoltarea aplicațiilor C/C++, dintre care amintim: compilatoarele GCC și Clang, respectiv sistemele de build CMake, Ninja și GNU Make. Astfel, ne limităm doar la asigurarea interacțiunii facile a sistemului cu cele mai cunoscute medii de dezvoltare moderne.

De asemenea, funcționarea sistemul distribuit presupune apartenența sistemelor de calcul participante la o rețea de calculatoare locală LAN. Scenariul funcționării într-o rețea

de noduri distribuite asupra unei arii geografice însemnate implică necesitatea implementării unor măsuri de securitate suplimentare pentru păstrarea integrității și confidențialității datelor transferate, iar această activitate depășește sfera de cercetare a lucrării.

Totodată, o altă condiție care este impusă pentru funcționarea o constituie uniformitatea mediului de rulare pentru nodurile participante, având la bază presupunerea că sistemele de calcul implicate sunt dotate cu aceeași versiune de sistem de operare, sisteme de build și compilatoare, iar arhitecturile hardware ale acestora sunt identice. Nu se studiază metode de partiționare ale colecției de noduri de calcul pe baza compatibilității dintre platformele de rulare ale acestora.

După implementarea sistemului de compilare distribuită, un alt obiectiv important al lucrării îl constituie evaluarea procesului de utilizare a sistemului, având în vedere analiza următoarelor aspecte cheie:

- Durata de build - se măsoară timpul de compilare pentru mai multe proiecte open-source arhicunoscute, atât folosind un unic nod de compilare, pentru a se determina măsura în care utilizarea sistemului introduce overhead, raportat la durata de compilare locală pe nodul respectiv, precum și folosind multiple noduri care contribuie la procesul de compilare, pentru a măsura, în mod evident, îmbunătățirile aduse la eficientizarea procesului de build
- Comportamentul determinist - în mod ideal, comportamentul procesului de build, folosind sistemul dezvoltat, trebuie să fie același la fiecare rulare, iar erorile sau warning-urile generate de către compilator trebuie să fie cât mai apropiate de cele rezultate în urma procesului de compilare locală
- Randamentul paralelizării - în funcție de natura proiectului care este supus procesului de compilare, sistemul de compilare distribuită aduce grade diferite de îmbunătățire a timpilor de build. În scenariul ideal, structura proiectului permite o micșorare a timpului de compilare direct proporțională cu numărul de sarcini de compilare paralelă oferite de totalitatea nodurilor participante, însă în realitate, unele proiecte nu se bucură de îmbunătățiri semnificative folosind o astfel de abordare
- Volumul de date transferat - distribuirea procesului de compilare către mai multe sisteme de calcul poate să implice transferul unor volume de date semnificative, în rețea, ceea ce introduce necesitatea accesului la resurse de rețea adecvate, pentru a nu deteriora eficiența procesului de build din cauza curențelor de date de capacitatea de transfer dintr-o perioadă de timp sau latența de inițiere a transferului

Capitolul 3

Studiu bibliografic/Stadiul actual în domeniu

În cadrul acestei secțiuni, vom prezenta câteva dintre soluțiile deja existente pentru a distribui sarcina de compilare a unui proiect asupra mai multor sisteme de calcul ce comunică prin intermediul unei rețele.

3.1 Distcc

Distcc[9] oferă posibilitatea de a distribui sarcina de compilare între mai multe sisteme ce comunică în rețea, fără a fi nevoie ca sistemele participante să partajeze același sistem de fișiere, fără sincronizare de clock-uri sau să aibă acces la aceeași versiune a codului sursă.

Distcc funcționează prin distribuirea sarcinilor de lucru de la un sistem client, către nodurile server, care vor realiza compilarea. Inițial, se analizează comanda de compilare executată de către client, pentru a determina dacă sarcina poate fi distribuită, se alege nodurile care să deservească cererea, apoi se rulează local preprocesorul asupra fișierelor ce vor fi supuse compilării. Ulterior, nodurile server primesc request-uri ce conțin fișierele sursă preprocesate și returnează către client fișierele obiect rezultate în urma compilării sau o listă cu erorile de compilare, în cazul unui eșec.

Aplicația client este invocată ca un wrapper pentru compilator-ul obișnuit (GCC de obicei), de către script-urile de build rulate de **GNU Make**. Comunicare între client-ul Distcc și nodurile server se realizează prin protocoalele de rețea **TCP** sau **SSH**, folosind un set simplu de mesaje pentru a trimite cereri de compilare: mesajul pentru cerere conține versiunea de protocol utilizată, argumentele din linia de comandă pentru invocarea compilatorului și fișierele sursă, în timp ce răspunsul unui nod server returnează fișierele obiect aferente.

Distcc oferă posibilitate ca mesajele interschimbate între nodurile client și server să fie comprimate înainte de trimitere, pentru a reduce dimensiunea volumului de date tri-

mis, folosind metoda de comprimare **LZO**[10], care se concentrează asupra obținerii unei viteze cât mai bune pentru operația de decompresie. Micșorarea volumului de date folosind compresia, îmbunătățește throughput-ul de cereri transmise între participanții sistemului Distcc.

Planificarea pentru lansarea noilor sarcini de compilare folosește un mecanism simplu, prin care fiecare participant păstrează un fișier de stare, iar suprasolicitarea unui nod este împiedicată prin definirea unui număr limită de job-uri de compilare care pot fi soluționate simultan, care ia o valoare dependentă de numărul de procesoare fizice de care dispune fiecare nod. La inițializare, fiecare nod pornește numărul maxim de procese Distcc-server, care ulterior așteaptă inițierea unei conexiuni de către un client care va solicita compilarea. Pentru a putea coordona activitatea proceselor server care rulează simultan pe fiecare nod, se folosesc fișiere lock Unix, care sunt eliberate automat de către un proces în momentul terminării sale.

Un dezavantaj al sistemului distribuit Distcc îl constituie faptul că nu există niciun mecanism de estimare a timpului de compilare pentru un nod, deoarece această valoare nu este dependentă doar de dimensiunea codului sursă, ci și de aspecte precum complexitatea codului sursă sau specificațiile hardware ale nodului respectiv. Așadar, pătrunderea unui nod foarte lent în rețeaua de servere Distcc, poate încetini semnificativ procesul de build, deoarece nodul solicitant va trebui să aștepte ca fiecare nod server să își finalizeze sarcina, înainte de a primi fișierele obiect și de a invoca linker-ul.

Totodată, un inconvenient care nu trebuie neglijat este faptul că Distcc este o aplicație care rulează exclusiv pentru sisteme de operare de tip **Linux**, ceea ce nu rezolvă problema compilării distribuite pentru **Windows**.

La data prezentării inițiale a Distcc, autorul afirmă că micșorarea timpilor de build este o valoare ce se modifică liniar, odată cu numărul de core-uri de procesare implicate, cel puțin pentru un număr mic de core-uri (pentru 3 calculatoare, build-ul a fost de 2.8 ori mai rapid). În ceea ce privește scalabilitatea sistemului Distcc, trebuie luat în calcul că dintre toate sarcinile aferente procesului de build: execuție Makefile, preprocesare, compilare, linking, doar compilarea este distribuită la mai multe noduri [11]. Pentru a determina care este limita de paralelizare a procesului de build, trebuie consultată Legea lui Amdahl, vizibilă în Figura 3.1:

$$speedup_{max} = \frac{1}{f + \frac{1-f}{P}}.$$

Figura 3.1: Legea lui Amdahl

Chiar dacă numărul de procesoare **P** tinde spre infinit, aportul de performanță este limitat de porțiunea din proces ce nu poate fi paralelizată [11].

Figura 3.2 exemplifică modul de funcționare al Distcc.

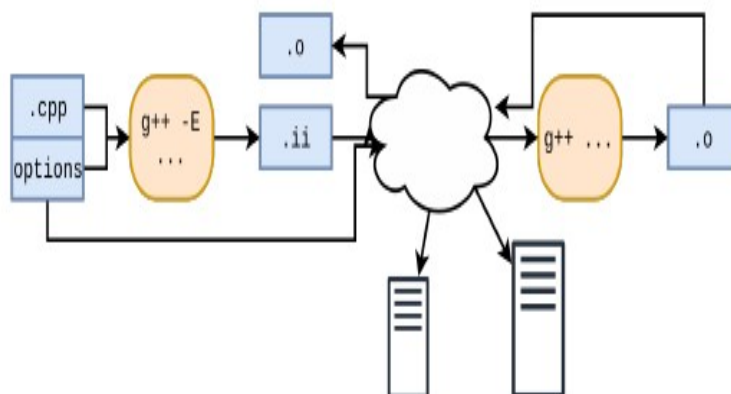


Figura 3.2: Funcționarea Distcc [12]

3.2 Ccache

Deși nu reprezintă un sistem distribuit, **Ccache**[13] este un mecanism caching pentru rezultatul compilării de fișiere sursă foarte des folosit alături de metoda descrisă anterior, Distcc, pentru a micșora și mai mult durata de build distribuit. Această combinație dintre **Distcc** și **Ccache** a fost studiată atent la Institutul CERN[12], întrucât procesul de build inițial pentru un proiect poate ajunge la 5 ore.

Sistemele tradiționale de build, precum **GNU Make** și **Ninja**, analizează timestamp-ul ultimei modificări a unui fișier sursă pentru a determina dacă compilatorul va fi invocat asupra unui fișier sursă sau nu. Problema apare în momentul schimbării de branch pentru sisteme de versionare precum **Git**, întrucât timestamp-ul fișierelor sursă își actualizează valoarea chiar dacă, conținutul nu se modifică, evenimentul fiind de natură să determine o recompilare completă a întregului proiect.

O proprietate interesantă a procesului de build pentru limbajele **C/C++** o constituie faptul că output-ul este complet determinat de conținutul surselor preprocesate și de argumentele de rulare ale compilatorului. Această proprietate este utilizată de către Ccache pentru a stoca output-ul compilării pentru un anumit fișier sursă, iar mai apoi să îl redea, în momentul începerii unui nou proces de build, evitând astfel apelarea inutilă a compilatorului pentru un fișier ce nu a fost modificat.

Sarcina principală pe care Ccache o îndeplinește o reprezintă determinarea existenței unui fișier object file asociat fișierului sursă de intrare, prin calcularea unei valori de dispersie bazată pe conținutul fișierului sursă și a flag-urilor folosite la compilarea inițială.

Ccache suportă mai multe moduri de operare, cel mai simplu dintre acestea fiind numit **preprocessor mode**, care presupune rularea preprocesorului asupra fișierului sursă

de intrare, iar mai apoi calcularea valorii hash pe baza conținutului unității de translație rezultate [14]. Dezavantajul acestui mod de funcționare este necesitatea de a apela întotdeauna preprocesorul asupra fișierului sursă de intrare, ceea ce încetinește lookup-ul.

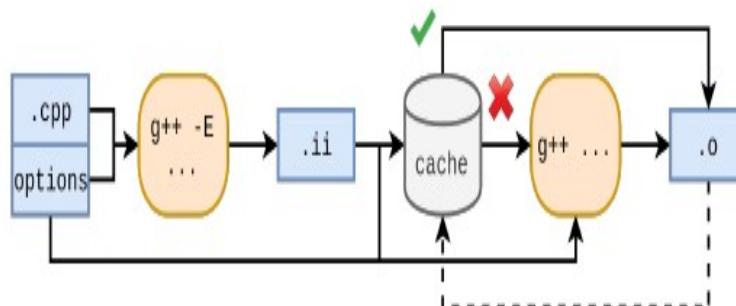


Figura 3.3: Ccache preprocessor mode [12]

De asemenea, Ccache poate funcționa într-un mod care evită pe cât posibil rularea preprocesorului, această acțiune fiind realizată doar în eventualitatea unui **cache miss**. În cadrul acestui mod de operare, valoarea hash este calculată direct pe baza fișierului sursă, fără preprocesare, iar valoarea este inclusă în cadrul unui fișier **manifest**, care conține referințe către rezultatele precedente al compilării (fișiere obiect, dependențe), care au fost produse pentru același hash, căi către fișierele header accesate la timpul compilării și valori checksum pentru fișierele incluse. În cazul unui cache miss, este necesară recalcularea valorii hash pentru fișierul de intrare și actualizarea manifest-ului.

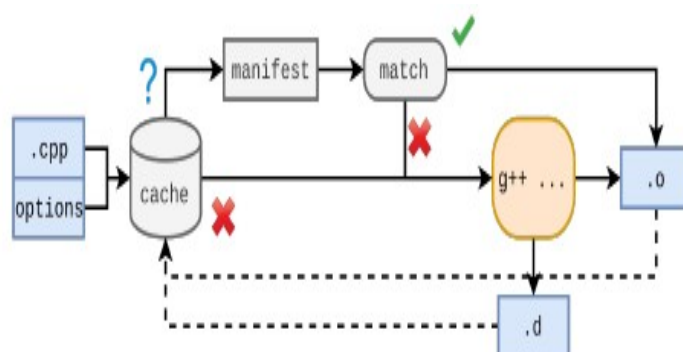


Figura 3.4: Ccache direct mode [12]

În cazul folosirii Ccache împreună cu sistemul distribuit Distcc, apare problema evidentă: cache-ul de compilare nu este distribuit pentru toate nodurile rețelei, ci fiecare nod participant are acces doar la cache-ul său local.

3.3 Sccache

Sscache[15] reprezintă varianta distribuită ca sistemului Ccache, care adaptează cache-ul inițial pentru a putea fi găzduit în cadrul unui serviciu cloud (sau pur și simplu remote) de stocare, precum: Amazon S3, Redis sau Cloudflare R2, ceea ce soluționează nevoia ca fiecare nod participant la procesul de compilare să dețină un cache local de fișiere obiect, care conține înregistrări doar pentru datele de intrare prelucrate de nodul respectiv.

Această soluție oferă un mecanism wrapper peste Ccache, astfel încât accesul la datele stocate remote să fie reglementat într-o manieră client-server [16]. Spre deosebire de Ccache, pentru calcularea unei valori hash se invocă întotdeauna preprocesorul, având astfel un comportament similar cu modul de operare **preprocessor mode**, dar se face offloading pentru volumul de date.

3.4 DiSCo

Dacă în cazul utilizării Distcc, singura dintre sarcinile din procesul de build care este paralelizată, este invocarea compilatorului asupra fișierelor preprocesate, **DiSCo** [17] propune o abordare inedită, în care preprocesarea, compilarea și link-editarea sunt efectuate în mod paralel, utilizând noduri remote.

Distcc utilizează schema de preprocesare locală, mult mai rudimentară, prin care nodul care solicită compilarea este singurul responsabil pentru realizarea etapei. Întrucât toată operațiunea are loc local, nu există problema ca un fișier de intrare să nu fie valabil, însă impune un cost însemnat asupra scalabilității sistemului. Pe de altă parte, preprocesarea remote, abordare aleasă de **DiSCo**, micșorează durata preprocesării, însă volumul de date tranzacționat între nodurile sistemului este mai mare, deoarece nu se mai trimite prin rețea doar fișierul preprocesat, ci atât fișierul sursă, cât și toate fișierele header asociate. O abordare asemănătoare adoptă și sistemul **IncrediBuild**[18], despre care nu sunt cunoscute multe detalii de funcționare, întrucât este o soluție comercială.

În ceea ce privește arhitectura sistemului de compilare distribuită **DiSCo**, la baza acestuia se află o serie de mai multe componente:

- Command Scheduler - modul care parsează conținutul unui **Makefile** pentru a determina potențialele dependențe dintre comenzile individuale de compilare, iar în funcție de relația dintre aceste comenzi, entitățile independente sunt grupate în blocuri de comenzi, elementele unui bloc fiind executate în paralel, iar mai multe blocuri de comenzi consecutive sunt executate secvențial; totodată, modulul command scheduler folosește o metodă de planificare statică pentru a determina spre ce nod al sistemului să fie trimis un anumit bloc de comenzi
- Connection Manager - folosește un mecanism de tip thread pool pentru a deschide sau termina conexiuni cu nodurile adiacente, folosite pentru comunicarea statusului pentru un proces de compilare în desfășurare [19]

- Compilation Manager - modul responsabil pentru executarea etapelor procesului de compilare
- Network Drive Manager - gestionează transferul de fișiere în mod implicit între nodurile sistemului și mediază interacțiunea cu **H-Cache-ul** nodului, responsabil pentru stocarea eficientă de fișiere header

Figura 3.5 ilustrează arhitectura sistemului distribuit de compilare DiSCo.

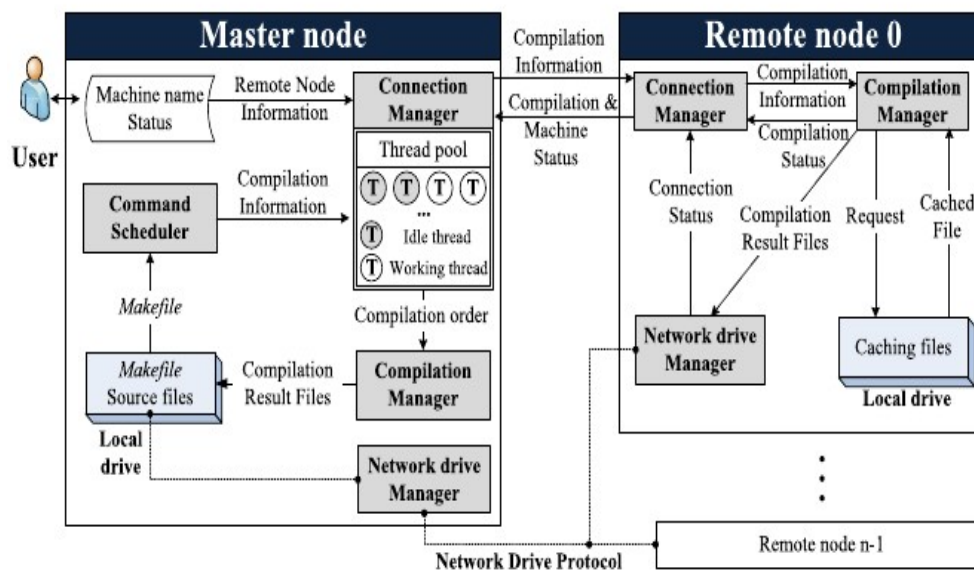


Figura 3.5: Arhitectura Disco [17]

Capitolul 4

Prezentarea proiectului

Acest capitol al lucrării are ca scop prezentarea principalelor concepte tehnice și arhitecturale care stau la baza implementării sistemului de compilare distribuită, descrierea deciziilor de proiectare care modelează structura sistemului și a soluțiilor practice propuse pentru etapa implementare. Sunt analizate în amănunt tehnologiile utilizate, pentru a crea o imagine de ansamblu a modului în care acestea influențează arhitectura sistemului.

4.1 Fundamentare teoretică

4.1.1 Sisteme distribuite

Abordarea calculului distribuit presupune utilizarea resurselor provenite de la mai multe sisteme de calcul interconectate pentru a contribui la rezolvarea sarcinii, în mod colaborativ. Astfel, dificultatea sarcinii nu rămâne rezervată unui singur sistem de calcul, ci este partiționată și distribuită în mod eficient către mai multe sisteme, pe baza metodelor de balansare a sarcinii. Fiecare dintre nodurile participante își pune la dispoziție atât capacitatea de calcul, precum și resursele de memorie, iar comunicarea cu celelalte noduri participante, în scopul coordonării, se realizează de cele mai multe ori în cadrul unei rețele de calculatoare [20].

Astfel, paradigma de calcul distribuit oferă avantaje semnificative, în ceea ce privește randamentul de procesare a datelor și asigurarea desiponibilității serviciilor, către utilizatorii sistemului:

- Toleranță la eșecuri - prin mecanisme de monitorizare periodică a stării de disponibilitate a nodurilor din sistem, se pot remedia situațiile în care una dintre cererile ce trebuie soluționate nu poate fi deservită din cauza indisponibilității temporare a unui nod implicat, întrucât nodurile redundate ale sistemului pot prelua sarcina nefinalizată
- Folosirea eficientă a resurselor neutilizate - în general, un computer obișnuit poate să se afle în stare de inactivitate cam în 70% din timpul total de funcționare [20],

ceea ce înseamnă că aceste resurse computaționale nefolosite pot fi agregate eficient în cadrul unui sistem distribuit pentru a rezolva sarcini costisitoare, a căror realizare nu ar fi fezabilă folosind un singur sistem de calcul

- Balansarea volumului de lucru - în cazul în care un nod solitar din cadrul sistemului distribuit nu face față volumului de lucru, alte noduri de calcul pot fi alocate în mod dinamic pentru a prelua din sarcini, fără a introduce ineficiențe legate de suprasaturarea resurselor unui anumit nod
- Scalabilitatea - limitările introduse de utilizarea unui singur sistem de calcul, al cărui potențial de performanță maximă este finit, pot fi mitigate prin introducerea de mai multe noduri participante, în funcție de nevoi
- Distribuirea geografică convenabilă - întrucât nodurile participante ale sistemului comunică prin intermediul rețelelor de calculatoare, distribuirea nodurilor asupra unei arii geografice de mare întindere devine o posibilitate, iar pentru fiecare utilizator al sistemului se pot identifica resursele care sunt cel mai apropiate și implicit, cele mai facil de accesat, pentru a satisface solicitările

Pe de altă parte, implementarea paradigmei de calcul distribuit implică dezavantajul creșterii semnificative a gradului de complexitate pentru realizarea unui sistem fiabil, din cauza necesității de a asigurarea un mediu de comunicare stabil și performant, dar și a garanției că sistemul are o stare globală consistentă. Pentru simplul fapt că informațiile legate de starea sistemului sunt împărțite pe multiple noduri, există riscul să apară inconsistențe între informațiile reținute de fiecare nod [21].

De asemenea, idealul ca un sistem distribuit să se afle permanent într-o stare adecvată de funcționare se întemeiază pe asumții care nu sunt tot timpul adevărate:

- Lățimea de bandă a canalelor de comunicație dintre noduri este infinită
- Structura rețelei și a nodurilor este omogenă
- Rețeaua este sigură
- Rețeaua este tot timpul disponibilă
- Latența pentru transferurile de date dintre noduri este neglijabilă

În încheierea acestei secțiuni, oferim figura 4.1, pentru a ilustra structura generală a unui sistem distribuit, în care mai multe noduri comunică într-o rețea cu topologie **"bus"**.

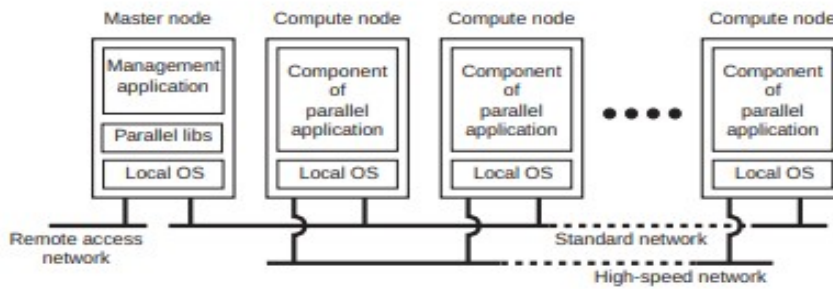


Figura 4.1: Sistem distribuit [21]

4.1.2 Mecanisme de concurență și paralelism

Una dintre problemele pe care orice sistem distribuit trebuie să le rezolve o reprezintă asigurarea capacității de comunicare simultană între un număr mare de noduri ale sistemului, pentru a oferi posibilitatea de scalare prin adăugarea de noduri suplimentare și a garanta că schimbul de informații dintre oricare două noduri nu trebuie să fie blocat până la finalizarea altor sarcini de calcul sau comunicare.

Având în vedere acest fapt, am ales să introduc în lucrare o secțiune ce prezintă fundamentele teoretice legate de asigurarea concurenței, în contextul dezvoltării sistemelor distribuite.

Concurența constituie capacitatea unui sistem de a executa mai multe sarcini în mod simultan. Până la momentul apariției sistemelor de calcul cu procesoare dotate cu mai multe nuclee, se crea impresia că mai multe sarcini de procesare erau soluționate în paralel prin schimbarea frecventă a contextului de execuție pentru sarcinile aflate în desfășurare [22]. În urma evoluției continue a resurselor hardware, au apărut mijloace multiple de asigurare a execuției concurente sau paralele.

Figura 4.2 are rolul de a ilustra diferența fundamentală dintre execuția concurentă și cea paralelă.

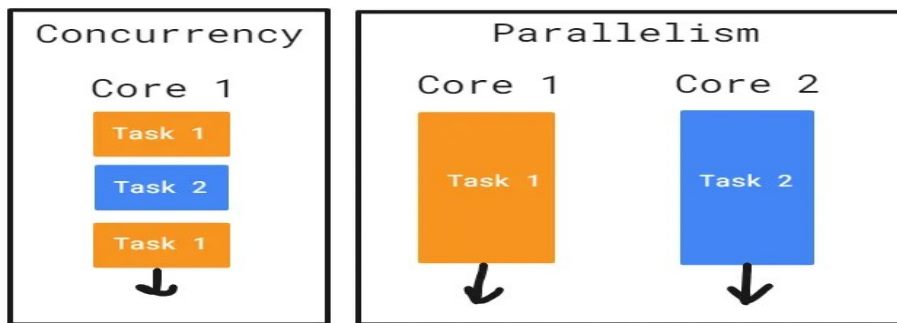


Figura 4.2: Concurență vs. paralelism [23]

Așadar, concurența presupune ca sistemul să asigure evoluția mai multor sarcini independente, care sunt active în același interval de timp și poate fi obținută prin comutare de context, fără a fi necesară utilizarea unui procesor cu mai multe nuclee sau a mai multor sisteme de calcul. Spre exemplu, concurența task-urilor este foarte utilă în momentul în care trebuie să așteptăm după un eveniment, fără a bloca execuția pentru restul programului. Mai sugestiv, toate aplicațiile server actuale trebuie să rezolve problema **C10K** [24], adică să asigure scalabilitatea aplicației pentru un număr mai mare de 10000 de conexiuni de rețea simultane, fără să epuizeze resursele computaționale. De cele mai multe ori, asigurarea concurenței este suficientă pentru a nu bloca firul principal de execuție al programului când se așteaptă finalizarea unei operațiuni **I/O** blocante ("I/O-bound"[22]).

Pe de altă parte, paralelismul oferă posibilitatea de a executa, în același moment discret de timp, două sau mai multe sarcini computaționale, fără a fi nevoie de executarea unei porțiuni din fiecare sarcină în mod secvențial pentru o perioadă redusă de timp. Principalul caz de utilizare al execuției paralele îl constituie împărțirea volumului de muncă între mai multe entități, cu scopul de a reduce durata de timp necesară pentru a finaliza procesarea ("CPU-bound"[22]).

În secțiunile următoare vom explica funcționarea câtorva dintre mecanismele de concurență și paralelism utilizate pentru dezvoltarea sistemului de compilare distribuită.

Thread-uri

Folosirea de multiple thread-uri[25] reprezintă o soluție eficientă pentru a rula mai multe task-uri în mod concurrent, în interiorul unui singur proces aflat în execuție. Una dintre diferențele principale dintre conceptul de thread și cel de proces, o constituie faptul că în vreme ce un proces își gestionează spațiul de adrese de memorie și resurse în mod total independent față de restul proceselor, mai multe thread-uri partajează aceleași resurse, ce aparțin procesului părinte.

Spre deosebire de procese, comunicarea între mai multe thread-uri active este mai facilă și mai eficientă, datorită spațiului de memorie partajat. De asemenea, împărțirea spațiului de memorie introduce necesitatea folosirii unor mecanisme de sincronizare între

firele de execuție (semafoare, excludere mutuală), pentru a împiedica coruperea datelor, datorată manipulării simultane a resurselor partajate, de către mai multe thread-uri care rulează în paralel.

Așadar printre cele mai utilizate mecanisme de sincronizare[22] pentru a asigura accesul sigur la resursele partajate, în contextul rulării în paralel a mai multor fire de execuție, le regăsim pe următoarele:

- **Mutex** - se utilizează pentru a asigura accesul exclusiv al unui singur thread la o porțiune critică din codul aplicației prin blocarea mutex-ului de către thread-ul care pătrunde în secțiunea critică. Celelalte thread-uri care urmează să execute codul din secțiunea protejată, trebuie să aștepte până când primul thread deblochează accesul la mutex
- **Semafor** - reprezintă o generalizare a conceptului de mutex, căci semaforul gestionează un contor numeric atomic, care descrie câte thread-uri se pot afla în regiunea de cod critică în mod simultan. Fiecare thread interacționează cu semaforul prin incrementarea sau decrementarea contorului acestuia, pentru a semnaliza celelalte thread-uri în legătură cu modificarea unei stări sau să aștepte ca alte thread-uri să părăsească regiunea critică. Folosind semafoare, se poate permite ca mai multe thread-uri să ruleze simultan cod din regiunea critică, până la un număr maxim de thread-uri
- **Variabila condițională** - mecanism de semnalizare între thread-uri care poate determina anumite fire de execuție să își suspende activitatea până la îndeplinirea unei condiții logice complexe. Thread-urile active pot să trimită semnale către variabila condițională, pentru a comunica thread-urilor care așteaptă, să își reia execuția

În cazul utilizării greșite a mecanismelor de sincronizare prezentate anterior, rularea mai multor fire de execuție în paralel determină apariția unor situații care deteriorează performanța aplicației și corectitudinea rezultatelor furnizate, posibil în mod nedeterminist:

- **Deadlock** - situație care apare când două sau mai multe thread-uri aflate în execuție se interblochează reciproc, întrucât fiecare dintre thread-urile în cauză reușește să obțină acces exclusiv la o resursă necesară pentru execuția unui alt thread, însă la rândul său, așteaptă ca accesul exclusiv la o resursă să fie eliberată. Această situație duce de obicei la blocarea pentru o perioadă nedeterminată de timp a thread-urilor aplicației
- **Livelock** - mai multe thread-uri rămân active pentru un timp nedeterminat, fără posibilitate de progres, întrucât încearcă fără succes să obțină acces la resurse
- **Înfometare (starvation)** - situație în care anumite thread-uri nu pot să își continue execuția, deoarece restul thread-urilor rulează în permanență și nu cedează accesul la resursele partajate după care așteaptă thread-urile înfometate

- Race condition - mai multe thread-uri accesează și încearcă să modifice starea unor resurse partajate în mod simultan, ceea ce generează stări impredictibile și eronate ale resurselor ce au fost modificate concurrent

Un alt aspect care merită menționat referitor la modul de funcționarea al thread-urilor, care este luat în considerare și în proiectarea sistemului de compilare distribuită, îl constituie faptul că, operațiunea de creare și distrugere repetată a unui număr mare de thread-uri, de fiecare dată când o sarcină trebuie soluționată concurrent, duce la pierderi semnificative de performanță la rulare. Aceste pierderi de performanță se datorează apariției foarte frecvente a evenimentelor de comutare de context între threaduri și a faptului că un număr excesiv de thread-uri concurează să obțină acces asupra resurselor partajate.

Pentru a mitiga dezavantajele generate de crearea unui thread nou pentru fiecare cerere ce trebuie soluționată, se poate folosi metoda **"Eager Spawning"**[26], care presupune inițializarea unui număr predefinit de thread-uri la momentul pornirii aplicației. Aceste thread-uri sunt adăugate la o colecție, numită **"thread pool"**, responsabilă să preia execuția cererilor generate de utilizatori, fără a se aștepta după inițializare de thread-uri noi sau după comutări de context. Pool-ul de thread-uri poate să fie de dimensiune fixă sau redimensionat în mod dinamic în timpul rulării, pe baza numărului total de cereri în așteptare sau a capabilităților hardware ale sistemului de calcul care rulează aplicația.

Thread-urile care fac parte din colecție așteaptă inactive în perioadele în care nu există sarcini de soluționat, pentru a nu irosi resurse. În momentul în care este înregistrată o sarcină nouă, unul dintre thread-urile inactive este desemnat pentru a realiza execuția, iar în situația în care toate thread-urile din pool sunt deja ocupate, atunci sarcina este nevoită să aștepte până la eliberarea unui thread.

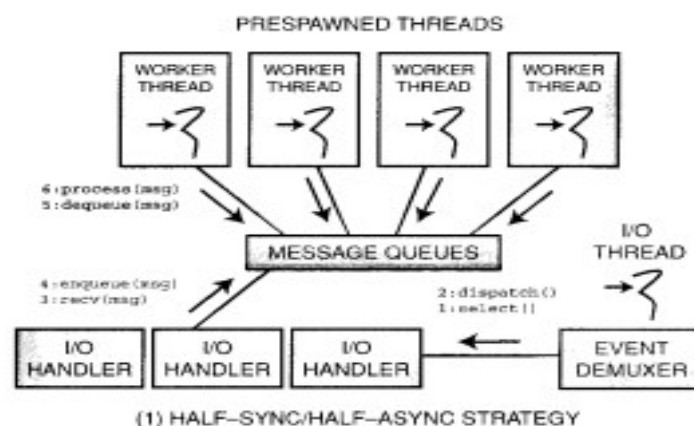


Figura 4.3: Concurență vs. paralelism [26]

Corutine

Rularea unei funcții obișnuite presupune execuția codului asociat de la începutul porțiunii de cod asociate, iar ieșirea din corpul funcției are loc o singură dată, moment în care apelul funcției este finalizat. O funcție nu stochează informații persistente de stare pentru rulare, motiv pentru care fiecare apel al funcției începe de la începutul blocului de cod asociat.

Corutinelor[27] reprezintă funcții care își pot sista temporar execuția când ajung la locații specifice din cod, iar execuția poate fi reluată ulterior de la locațiile în care a avut loc întreruperea. Pe durata întreruperii execuției, implementarea salvează informații de stare ale funcției înainte de suspendare, precum conținutul regiștrilor și valorile variabilelor locale de pe stivă. Capacitatea de a opri și a relua execuția înainte de finalizarea apelului oferă numeroase avantaje practice: spre exemplu, o corutină care trebuie să aștepte finalizarea unui apel blocant pentru a progresa poate să își suspende execuția pentru a permite altor funcții să ruleze, fără a bloca firul principal de execuție al aplicației. Astfel, corutinelor reprezintă un mecanism util pentru a asigura concurența aplicației, folosind un număr restrâns de thread-uri sau chiar un unic thread, care permite corutinelor să ruleze cod în mod cooperativ, prin suspendarea și reluarea execuției la momentul în care toate resursele devin accesibile.

În funcție de maniera de implementare a conceptului de "corutină", corutinelor se împart în corutine cu stivă proprie sau fără stivă ("stackless"). Pentru o corutină cu stivă proprie, toate variabilele locale pot să fie stocate persistent fără ca cel ce apelează corutina să aibă acces la valori. Pentru corutinelor fără stivă, toate valorile de pe stiva funcției sunt eliberate, ca pentru o finalizare obișnuită a unui apel de funcție, iar informațiile legate de starea de execuție a corutinei trebuie copiate într-o zonă de memorie auxiliară până la reluarea execuției. Corutinelor fără stivă prezintă avantajul că salvarea stării la momentul suspendării nu este obligatorie, ceea ce evită execuția suplimentară a a procesului de copiere a datelor de stare, însă nu mai este permisă suspendarea execuției corutinei în orice locație a blocului de cod asociat.

Spre exemplu, standardul **C++ 20** implementează corutinelor fără stivă proprie[28], iar starea de execuție, variabilele locale și parametrii sunt stocate într-un Coroutine Stack Frame, care se alocă în mod dinamic pe heap. Figura 4.4 ilustrează informațiile care sunt reținute la momentul suspendării execuției.

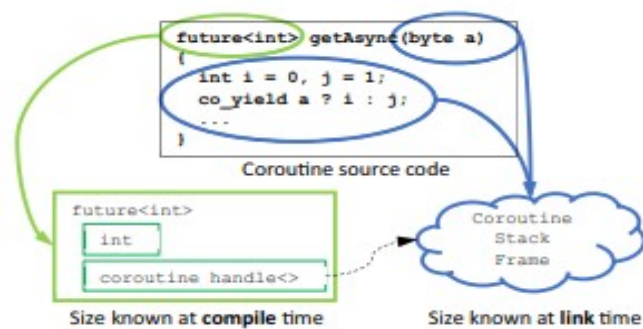


Figura 4.4: Coroutine stack frame [26]

4.1.3 Funcționarea procesului de compilare pentru C/C++

Întrucât **C** și **C++** fac parte din categoria limbajelor de programare compilate, codul sursă scris în aceste limbaje trebuie mai întâi supus unei transformări în cod mașină și împachetat într-un fișier executabil sau o librărie, pentru a putea fi rulat de către sistemul de calcul către care se adresează compilarea. Pentru a produce rezultatele finale, compilarea trebuie să parcurgă mai multe etape[3]:

- Preprocesarea - înainte să înceapă compilarea efectivă, directivele de preprocesare (de cele mai multe ori marcate cu "#", precum **#define**, **#include**) și macro-urile (aparitiile din cod ale simbolurilor declarate folosind **#define**) sunt expandate, prin înlocuirea cu valoarea lor efectivă: dacă spre exemplu, avem în codul sursă secvența **"#include "header.h"**, atunci preprocesorul va adăuga la codul sursă întreg conținutul fișierului "header.h"
- Compilarea - se efectuează transformarea codului sursă C/C++ în serii de instrucțiuni ale limbajului de asamblare specific arhitecturii pentru care se efectuează compilarea, iar secvențele invalide de cod sursă sunt semnalizate prin afișarea de erori și oprirea întreg procesului de compilare
- Asamblarea - se preiau fișierele cu instrucțiuni în limbaj de asamblare rezultate la pasul anterior și se utilizează ca date de intrare pentru asamblor, care produce fișierele obiect, populate cu seriile de instrucțiuni în cod mașină asociate programului în limbaj de asamblare
- Link-editarea - ultima etapă a procesului de compilare presupune agregarea tuturor fișierelor obiect rezultate în urma etapei de asamblarea pentru a produce librăria sau executabilul final, iar de asemenea, dacă programul utilizează librării externe, codul mașină asociat acestora este adăugat la executabilul generat

Entitățile componente ale unui proiect C/C++, respectiv artefactele rezultate în urmă desfășurării procesului de build sunt următoarele:

- Fișiere header - sunt marcate de cele mai multe ori folosind extensii precum **.h**, **.hh** sau **.hpp** și conțin definiții pentru funcțiile și tipurile de date utilizate în interiorul fișierelor sursă ale proiectului: structuri, clase, funcții, variabile. Fișierele header nu sunt compilate în mod independent, dar conținutul lor este înlocuit cu directivele **#include** asociate în urma preprocesării
- Fișiere sursă - sunt marcate cu extensiile **.c**, **.cc**, **.cpp** și conțin implementarea aplicației care se compilează, conținutul acestor influențând în mod direct funcționarea aplicației. Ele sunt transformate în fișiere obiect, ce conțin cod mașină, în urma procesului de compilare
- Fișiere obiect - marcate cu extensia **.o**, ele sunt rezultatul asamblării rezultatelor produse în faza de compilare și conțin cod mașină ce poate fi executat de arhitectura sistemului de calcul pentru care s-a comandat compilarea
- Fișiere executabile - există mai multe tipuri de fișiere executabile, în funcție de modul în care a fost comandată compilarea: programe ce se pot rula (**.exe**, **.out**), librării statice ce pot fi adăugate în etapa de link-editare a procesului de build pentru alt proiect cu scopul de a adăuga funcționalități suplimentare (**.a**, **.lib**) sau librării dinamice, care pot fi încărcate în mod partajat de către mai multe executabile în timpul rulării (**.dll**, **.so**)

În încheierea acestei secțiuni, oferim Figura 4.5 pentru a ilustra etapele procesului de compilare pentru fișiere sursă ale unui proiect C/C++.

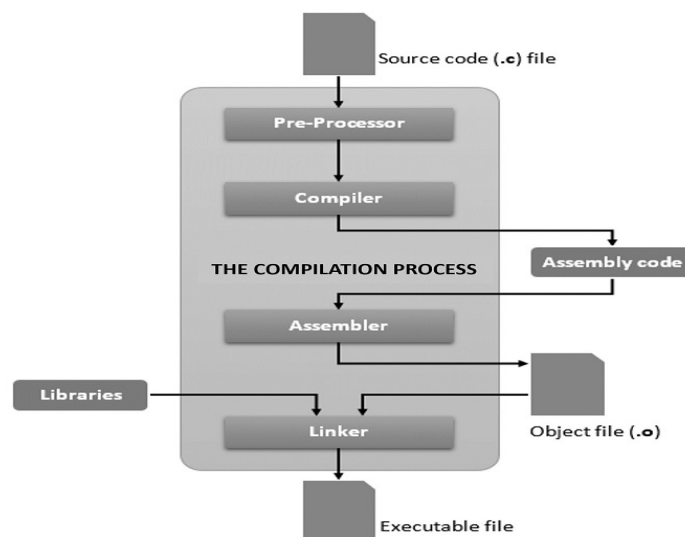


Figura 4.5: Compilare fișierelor sursă C/C++ [29]

4.1.4 Sisteme de build pentru C/C++

De cele mai multe ori, proiectele dezvoltate în C și C++ au în componență un număr foarte mare de fișiere sursă pe baza cărora se generează executabilul final, iar pe lângă numeroasele invocări ale compilatorului necesare în procesul de build al proiectului, se realizează o mulțime de operațiuni adiționale pentru a genera structuri de fișiere necesare pentru build, a localiza dependențele necesare sau a identifica care pași din procesul de build pot fi ignorați deoarece au fost efectuați deja cu succes. De cele mai multe ori, pentru a compila cu succes proiectul și a-l aduce într-o stare funcțională, este nevoie să se ruleze foarte multe comenzi diferite, ceea ce reprezintă o activitate aproape imposibilă dacă se dorește introducerea manuală.

Pentru a automatiza întreg procesul de build al proiectelor C/C++, se folosesc sisteme de build, al căror rol este cel de a transforma un limbaj de configurare expresiv și ușor de utilizat în secvența de comenzi ce vor fi rulate pentru produce artefacte în urma compilării proiectului.

Pe lângă faptul că utilizarea unui sistem de build scutește programatorul de la rularea manuală a comenzilor necesare, cele mai multe sisteme de build încearcă să reducă timpii de compilare ai proiectului determinând dacă fișierele sursă au suferit modificări de la ultima compilare, urmând să fie compilate doar fișierele ce au fost modificate. De asemenea, toate sistemele de build uzuale permit ca proiectul gestionat să fie compilat folosind opțiuni diverse de configurare, precum definirea unui anumit standard al limbajului C sau C++ care să fie folosit de compilator, prezența sau absența simbolurilor de debug, setarea nivelului de optimizare aplicat de compilator.

O particularitate comună a tuturor sistemelor de build pentru C/C++ o reprezintă faptul că permit definirea de mai multe configurații pe baza cărora să se realizeze compilarea, numite **"target"**[30]. În funcție de nevoile ce apar pe parcursul dezvoltării proiectului, proiectul va trebui compilat în configurații diverse, spre exemplu: build pentru toate configurațiile existente sau doar pentru o configurație anume, build ce injectează în sursele proiectului secvențe de cod folosite pentru măsurarea performanțelor la rulare, build pentru suita de teste asociată proiectului, și așa mai departe.

În continuarea vom descrie succint modul de funcționare pentru câteva dintre cele mai cunoscute sisteme de build folosite pentru limbajele C și C++: **GNU Make**[30] și **CMake**.

GNU Make[30]

Utilitarul **make** primește ca date de intrare script-uri **Makefile**, în care sunt plasate instrucțiuni ce descriu pașii de urmat pentru realiza procesul de build pentru un proiect. GNU Make reprezintă un sistem de build universal și poate fi folosit pentru orice fel de limbaj, nu doar pentru C sau C++. Script-ul Makefile conține o serie de reguli care specifică modul în care fiecare fișier sursă trebuie compilat, având în vedere o listă de dependențe asociată fișierului sursă. Un fișier sursă este recompilat doar în momentul în care el însuși sau una dintre dependențele sale au suferit modificări.

Rularea utilitarului **make** trebuie efectuată în interiorul unui director care conține script-ul **Makefile**, menționat precedent, iar dacă nu se furnizează niciun target ca argument din linia de comandă, atunci GNU Make va realiza build-ul pentru primul target întâlnit în cadrul scriptului, care de obicei este responsabil pentru compilarea întregului proiect.

Figura 4.6 reprezintă un script Makefile generic, folosit pentru a exemplifica ulterior structura unui astfel de script.

```
edit : main.o kbd.o command.o display.o \  
      insert.o search.o files.o utils.o  
      cc -o edit main.o kbd.o command.o display.o \  
          insert.o search.o files.o utils.o  
  
main.o : main.c defs.h  
      cc -c main.c  
kbd.o : kbd.c defs.h command.h  
      cc -c kbd.c  
command.o : command.c defs.h command.h  
      cc -c command.c  
display.o : display.c defs.h buffer.h  
      cc -c display.c  
insert.o : insert.c defs.h buffer.h  
      cc -c insert.c  
search.o : search.c defs.h buffer.h  
      cc -c search.c  
files.o : files.c defs.h buffer.h command.h  
      cc -c files.c  
utils.o : utils.c defs.h  
      cc -c utils.c  
clean :  
      rm edit main.o kbd.o command.o display.o \  
          insert.o search.o files.o utils.o
```

Figura 4.6: Makefile[31]

La baza oricărui script Makefile se află **reguli (rules)**, care specifică ce target-uri trebuie compilate, care sunt condițiile necesare (prerequisites) pentru a declanșa compilarea pentru target-uri, precum și comanda concretă care trebuie rulată pentru a compila target-urile sepcificate. O regulă are următorul format:

```
target: prerequisites  
... comenzi de compilare ...
```

Cele mai uzuale elemente ale oricărui script Makefile sunt următoarele:

- Targets - reprezintă rezultatul de ieșire ce trebuie creat în urma compilării, fiind în general asociate unor fișiere obiect sau executabile. Un caz de target special îl reprezintă target-urile phony, care sunt asociate cu rularea unor comenzi specifice: build pentru tot proiectul sau ștergerea artefactelor rezultate din build-urile anterioare

- Prerequisites - fişierele sau ţintele ce trebuie să existe sau să fi fost soluţionate deja, pentru a putea rula ţinta curentă. Dependinţele ce au suferit modificări sunt reactualizate, iar ţinta curentă este rulată din nou
- Lista de comenzi (Recipe) - comenzile din listă sunt rulate odată cu solicitarea unui build pentru ţinta părinte, iar lista poate conţine orice fel de comenzi care pot fi rulate în interiorul unui terminal
- Special targets - aceste target-uri au o semnificaţie specială şi de cele mai multe ori sunt asociate cu anumite seturi de operaţiuni, nu fişiere care trebuie compilate. Un target special este specificat folosind caracterul ”.” în faţa numelui (spre exemplu)
- Prefixuri de comenzi - prin utilizarea de caractere speciale, se poate altera modul de rulare pentru comenzile asociate unei reguli: ”-” - se ignoră erorile generate de rularea comenzii curente, ”+” - se forţează rularea comenzii respective, ”@” - nu se afişează în terminal fluxul de ieşire al comenzii

CMake[32]

CMake reprezintă un **meta-build system**, întrucât acesta transformă conţinutul script-urilor **CMakeLists.txt** în script-uri de configurare pentru alte tipuri de sisteme de build: GNU Make, Ninja, Autotools, MSBuild şi altele. CMake are ca rol automatizarea activităţilor de dezvoltare uzuale din cadrul unui proiect:

- Gestiunea dependenţelor
- Compilarea proiectului sub formă de executabil sau librărie
- Instalarea pe sistem a modulului rezultat în urma build-ului
- Generarea automată de documentaţie
- Rularea de teste

Funcţionarea sistemului de build **CMake** se bazează pe tool-uri externe pentru efectuarea operaţiunilor specificate în scriptul de build **CMakeLists.txt**, întrucât acest sistem de build are doar rol în orchestrarea activităţilor asociate procesului. Rularea procesului de build este împărţită în trei etape: configurarea, generarea proiectului şi compilarea efectivă.

În faza de **configurare**, se creează un director nou care va conţine informaţii legate de mediul în care se produce build-ul proiectului, precum compilatoarele şi utilitarele existente, arhitectura sistemului, existenţa unor librării folosite la linking, iar un program de test este compilat pentru a verifica că toolchain-ul funcţionează cum trebuie.

Pentru a se determina structura proiectului, dependenţele de compilare şi target-urile de compilare, se execută script-ul CMakeLists.txt. Informaţiile care au şanse mari

să rămână nealterate de la o rulare la alta sunt stocate într-un fișier **CMakeCache.txt**, pentru a economisi timp la rulările următoare.

Urmează faza de **generare** a proiectului, în care comenzile din script-ul CMakeLists.txt sunt traduse în alte script-uri aferente build system-ului configurat de la început: Ninja, GNU Make, MSBuild.

Faza de **build** produce rezultatele așteptate în urma compilării proiectului, căci este etapa responsabilă pentru rularea build system-ului corespunzător script-urilor produse în etapa de generare. Acum se apelează compilator-ul, linker-ul, utilitare pentru analiza codului sau generarea automată de documentație, precum și alte operațiuni configurate.

În finalul secțiunii, oferim Figura 4.7 pentru a exemplifica modul de funcționare pentru meta-build system-ul CMake.

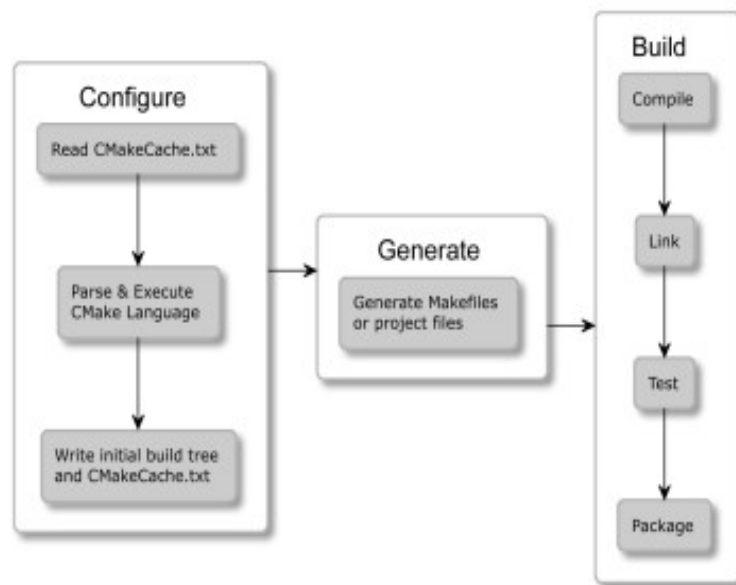


Figura 4.7: Etapele funcționării CMake[32]

4.2 Arhitectura conceptuală a sistemului

Această secțiune a lucrării are rolul de a explica în mod detaliat alegerile din spatele deciziilor arhitecturale care modelează structura și funcționarea sistemului de compilare distribuită. Se prezintă principalele module care intră în componența aplicației, alături de tehnologiile alese pentru realiza implementa funcționalitatea acestora.

Întrucât se dorește dezvoltarea unui sistem software care să preia cererile de compilare solicitate de către un sistem de calcul gazdă și să le distribuie în mod uniform către alte sisteme accesibile în rețea, pentru a putea compila cât mai multe fișiere sursă în regim paralel, am elaborat o listă minimală de cerințe funcționale ce trebuie implementate pentru elaborarea unui astfel de sistem:

- Interceptarea comenzilor de compilare - pentru a putea să distribuim către alte noduri din rețea sarcinile de compilare, trebuie să interceptăm fiecare apel către compilatorul clasic și să îl redirectionăm către un modul care va declanșa rularea comenzii respective pe unul dintre nodurile disponibile
- Parsarea comenzilor de compilare - în funcție de argumentele din linia de comandă care sunt folosite pentru a compila un anumit fișier sursă, anumite opțiuni furnizate pentru compilarea locală necesită ajustări pentru ca compilarea să se realizeze cu succes în mediul de rulare al dispozitivul remote care preia compilarea
- Elaborarea listei cu dependențele de compilare - fiecare fișier sursă are nevoie ca mulțimea de fișiere header pe care le include să fie expandate înainte de declanșare etapei de compilare, motiv pentru care, pe lângă fișierul sursă de compilat, la nodul remote trebuie să fie trimise și toate fișierele header care fac parte din lista de dependențe a acestuia
- Definirea unui protocol de comunicație - funcționarea sistemului se bazează pe interacțiunea dintre mai multe sisteme de calcul, prin intermediul unei rețele de calculatoare, deci este necesară proiectarea unui protocol de comunicare care să modeleze cât mai fidel interacțiunea dintre nodurile implicate: inițializarea unei sesiuni de compilare pentru un fișier, transmiterea dependențelor de compilare pentru fișierul respectiv, transmiterea rezultatelor compilării înapoi la nodul care a lansat solicitarea, semnalarea erorilor de comunicare și așa mai departe
- Distribuirea sarcinii pe noduri - se dorește implementarea unui mecanism care să publice date despre resursele disponibile, ce pot fi folosite pentru compilare, iar în baza acestor informații să se aleagă nodul care va deservei cererea de compilare
- Asigurarea compatibilității cu tool-urile existente - se dorește ca sistemul să fie ușor de integrat pentru a reduce durate de compilare a unui proiect care folosește compilatoare și sisteme build bine cunoscute în industrie, fără a fi nevoie să fie modificată structura proiectului sau a configurației de build pentru a putea beneficia de avantajele oferite de sistemul de compilare distribuită

- Asigurarea validității datelor - întrucât transferul tuturor dependențelor pentru un proiect C/C++ presupune trimiterea în rețea a unui volum însemnat de date, este necesar ca odată primite, validitatea datelor să poate fi verificată, pentru a avea o siguranță cât mai mare că operațiunea de compilare distribută va produce aceleași rezultate ca operațiunea de compilare locală

Principalul impuls care a determinat alegerea implementării unui sistem de compilare distribuită ca temă a lucrării îl constituie dorința de a explora posibilitatea de a reduce timpul de compilare pentru proiectul în care activez, la actualul loc de muncă. Organizația din care fac parte a optat pentru dotarea echipelor cu sisteme workstation performante (dotate cu procesoare cu 18 nuclee), care în marea majoritate a timpului nu sunt utilizate astfel încât să fie exploatată pe deplin capacitatea lor de calcul, iar proiectele dezvoltate în cadrul departamentului pot să depășească durate de zeci de minute pentru a finaliza procesul de build. Acest context modelează în mare parte constrângerile pe care le impunem în implementarea sistemului:

- Nodurile care fac parte din sistem sunt situate în aceeași rețea LAN
- Resursele computaționale ale nodurilor care participă la compilare nu diferă de la un nod la altul
- Mediul de rulare (sistem de operare, versiune tool-uri) este aproape identic pentru toate nodurile sistemului

Având în vedere cerințele funcționale stabilite anterior, am identificat modulele de bază care sunt responsabile pentru distribuirea și efectuarea comenzilor de compilare în cadrul sistemului:

- Interceptorul pentru comenzile de compilare - are ca rol captarea de apeluri locale ale compilatorului și transmiterea acestora către un alt modul care să realizeze operațiunile necesare pentru distribuirea compilării către nodurile disponibile
- Aplicația client - această aplicație va rula pe nodul care comandă compilarea și transformă comenzile de compilare primite de la interceptor în solicitări care sunt trimise către nodurile ce rulează aplicația server care declanșează compilarea, iar totodată orchestrează tot fluxul de comunicare cu nodurile server
- Aplicația server - va rula pe nodurile ca își pun la dispoziție resursele de calcul și are rolul de a soluționa cererile de compilare primite de la nodul care rulează aplicația client, precum să și stocheze dependențele primite de la nodurile client și fișierele obiect rezultate în urma compilării, care sunt returnate prin rețea către nodul solicitant

Figura 4.8 ilustrează într-o manieră simplificată interacțiunea dintre modulele sistemului de compilare distribuită.

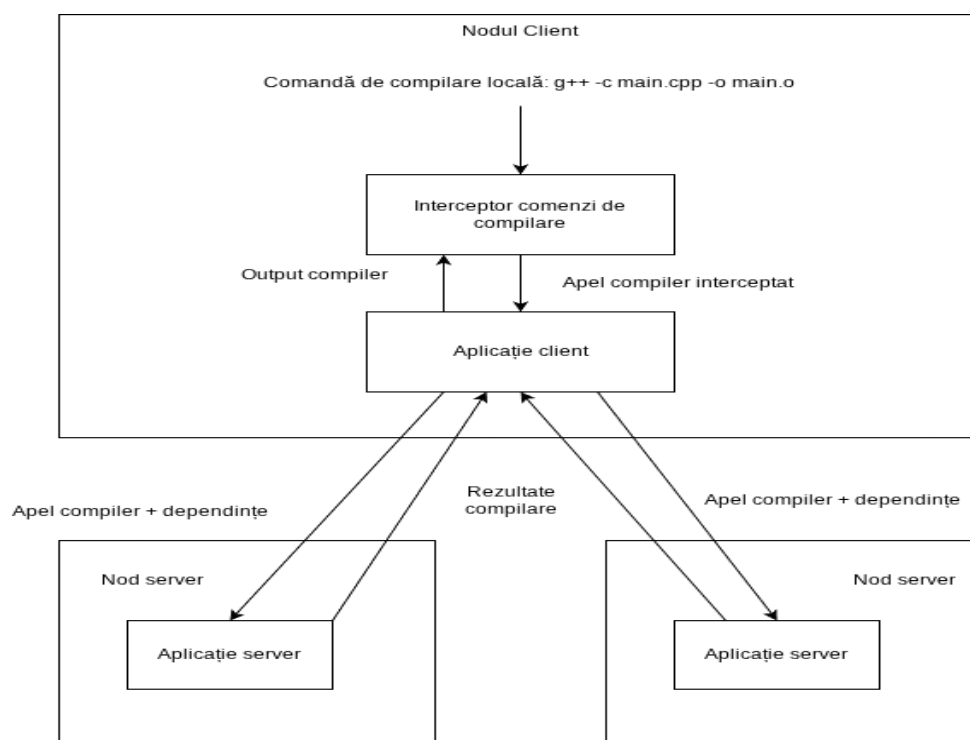


Figura 4.8: Diagrama conceptuală a sistemului de compilare distribuită

În continuare vom analiza alegerile arhitecturale pentru cele trei componente mari ale sistemului.

4.2.1 Interceptorul pentru comenzile de compilare

O primă observație care trebuie enunțată în legătură cu arhitectura mecanismului de interceptare a comenzilor de compilare o reprezintă faptul că funcționarea acestui modul nu trebuie să altereze apelurile de compilare generate de către toate proiectele găzduite pe nodul client, deoarece compilarea acestora folosind sistemul de compilare distribuită este opțională.

Astfel, este necesar să găsim o modalitate de a implica sistemul de compilare distribuită în procesul de build al unui proiect doar în cazul în care această alegere este setată în mod explicit, fără a deveni un comportament global. Pentru rezolvarea acestei probleme se poate apela la setarea unor anumite opțiuni de configurare pentru sistemul de build folosit de către proiectul cauză, astfel încât în locul comenzii obișnuite de compilare pentru un fișier sursă, să se apeleze comanda respectivă cu un prefix, care să redirectioneze comanda de compilare către un alt program.

Spre exemplu, această funcționalitate este prezentă în cadrul sistemului de build **CMake**, care permite specificarea parametrului **-DCMAKE_CXX_COMPILER_LAUNCHER**, la rularea pentru generarea inițială a structurii proiectului, pentru a alege un executabil care să fie folosit ca prefix pentru toate invocările de compiler. Se va rula comanda CMake pentru generarea proiectului:

```
cmake -S . -B ./build \
      -DCMAKE_BUILD_TYPE=Release \
      -DCMAKE_CXX_COMPILER_LAUNCHER="/distbuild_wrapper"
```

, iar fiecare apel către compilator va avea adăugat ca prefix executabilul respectiv:

```
distbuild_wrapper /usr/bin/c++ \
  -DPROJECT_EXPORTS \
  -DNDEBUG \
  -Iinclude \
  -std=c++20 \
  -O2 \
  -include build/cmake_pch.hxx \
  -c src/server.cpp \
  -o build/server.o
```

Această abordare este folosită și de către sistemul de compilare distribuită **Distcc**[33], despre care se menționează că un executabil este folosit ca wrapper pentru comenzile către compilatorul **GCC**, invocate prin rularea script-ului **Makefile**.

Executabilul wrapper pentru compiler, trebuie să imite comportamentul compilatorului, astfel încât sistemul de build care apelează compilatorul în mod normal să nu detecteze erori legate de funcționarea diferită a procesului de build. Pentru fiecare invocare activă a compilatorului, se va crea un nou proces al wrapper-ului, care este dator să implementeze următoarele caracteristici de comportament:

- Să deschidă o conexiune către aplicația client aflată în rulare
- Să trimită către aplicația client întreaga comandă de compilare
- Să trimită către aplicația client directorul curent, în care a fost invocat executabilul wrapper, deoarece în funcție de această locație aplicația client trebuie să realizeze transformări asupra path-urile transmise ca parametru în comanda de compilare
- Să nu își înceteze execuția până când rezultatul compilării fișierului nu este recepționat, întrucât la un apel adevărat, executabilul compilatorului rulează până când compilarea s-a finalizat
- Să afișeze toate erorile de compilare sau warning-urile aferente apelului de compilare asociat, odată ce acestea sunt primite de la aplicația client

Un alt aspect care merită discutat în continuare îl constituie modalitatea în care se petrece comunicarea dintre executabilul wrapper care interceptează comenzile de compilare și aplicația client. Trebuie luat în calcul faptul că întotdeauna aplicația client a sistemului va rula pe același nod care rulează și numeroasele instanțe ale executabilului wrapper (câte un proces al executabilului wrapper pentru fiecare apel de compiler activ), deci este foarte avantajoasă utilizarea tehnicilor de comunicare între procese (**Inter-Process Communication**).

UNIX domain sockets[34] pot fi utilizate pentru a facilita comunicarea dintre procese care rulează pe același computer. Este posibilă și utilizarea socket-urilor clasice care fac parte din **Internet domain**, însă comunicarea folosind socket-uri UNIX domain este mai eficientă: se bazează doar pe copierea de date, fără a fi nevoie să urmeze pașii de procesare ai unui protocol anume, deci fără utilizare de segmente header care trebuie adăugate la datele trimise, fără numere de secvențiere pentru mesaje și confirmări de primire a datelor. Spre deosebire de socket-urile obișnuite ce aparțin de Internet domain, care pot fi asociate cu o adresă IP și cu un port, socket-urile UNIX se asociază cu un path către un fișier de tipul **S_IFSOCK** cu numele respectiv.

Astfel, la nivelul aplicației client, există posibilitatea de a crea un socket UNIX, care să fie asociat unui path unic, iar toate instanțele de proces ale executabilului wrapper să stabilească conexiuni cu acest socket UNIX pentru a transfera comenzile de compilare și path-ul directorului din care s-a solicitat compilarea către aplicația client.

Figura 4.9 ilustrează comunicarea dintre instanțele de proces active ale wrapper-ului de compiler și aplicația client.

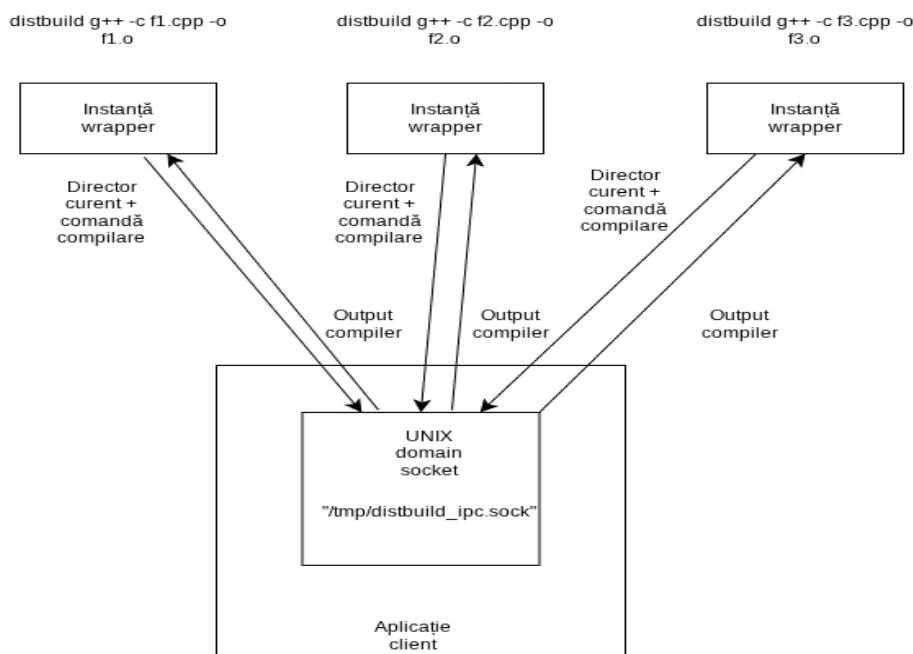


Figura 4.9: Comunicare cu aplicația client prin UNIX domain socket

4.2.2 Aplicația client

Înainte de a explica arhitectura aplicației client, trebuie să oferim câteva informații legate de modalitatea de paralelizare a compilării proiectelor, care afectează funcționalitățile aplicației client:

- Pentru a se putea realiza transformarea unui fișier cu cod sursă C/C++ într-un fișier obiect, care conține cod mașină, este nevoie ca la momentul compilării să fie valabile toate fișierele header care sunt incluse de către fișierul sursă respectiv, iar aceasta cerință se extinde și asupra fișierelor header incluse, în mod recursiv
- Pentru a se efectua operațiunea de link-editare și creare a fișierului executabil, fie că acesta este un executabil de sine stătător sau o librărie, este nevoie a toate fișierele obiect provenite în urma compilării proiectului să fie disponibile

Așadar, principiul de funcționare al aplicației client se bazează pe distribuirea sarcinii de compilare a fiecărui fișier sursă din proiect către un nod server, iar fișierele obiect primite de la nodurile server sunt folosite de către client pentru a realiza în mod local operațiunea de link-editare a artefactelor finale. Link-editarea este soluționată local, pentru că necesitatea de a avea acces la toate fișierele obiect ale proiectului o face să fie o operație foarte dificil de paralelizat. În cazul în care compilarea unui fișier sursă pe un nod server eșuează cu o eroare de compilare, atunci compilarea pentru fișierul respectiv este reluată de către aplicația client, deoarece există anumite situații în care unele fișiere sursă nu pot să fie compilate în afara mediului de lucru original (spre exemplu, fișierul sursă conține directive **#include** care încearcă includerea unor fișiere din path-uri absolute, care nu există pe nodul server).

În continuare, vom prezenta lista reponsabilităților concrete pe care trebuie să le îndeplinească aplicația client, a căror metodă de integrare o vom analiza în amănunt:

- Să stabilească conexiuni folosind socket-uri UNIX domain cu toate instanțele wrapper-ului de compiler și să returneze rezultatul operațiunilor de compilare către acestea
- Să realizeze parsarea apelurilor de compiler și să ajusteze parametrii astfel încât comandă să poată fi rulată de către un nod server. De asemenea, trebuie extrase informații necesare pentru determinarea dependențelor fișierului sursă supus compilării
- Să inițieze și gestioneze conexiuni multiple cu nodurile server, în mod concurent, și să transmită solicitările de compilare către nodurile server cu resurse disponibile, iar rezultatele de compilare să fie centralizate concurent
- Să realizeze operațiunea de link-editare în mod local
- Să termine sesiunile de comunicare eșuate cu server-ul, în cazul identificării unui defect în timpul schimbului de mesaje

- În cazul unui eșec de compilare al unui nod server, să încerce să efectueze operațiunea de compilare în mod local

Parsarea comenzilor de compilare

Comanda de compilare primită de la o instanță a wrapper-ului de compiler conține toate informațiile de care este nevoie pentru a genera un fișier obiect din codul sursă furnizat ca intrare. Printre informațiile importante care se regăsesc în cadrul unei astfel de comenzi, amintim: tipul de compilator utilizat, path-ul cu locația fișierului sursă de compilat, directoarele suplimentare în care sunt situate fișiere header incluse prin directiva **#include**, opțiuni generale de compilare (nivelul de optimizare, introducerea simbolurilor de debug), constante adiționale care să fie introduse în etapa de preprocesare, locația la care să fie plasat fișierul obiect rezultat.

În cadrul sistemului de compilare distribuită, informațiile extrase din interiorul comenzii de compilare primite sunt folosite pentru a genera corect lista de dependențe header a fișierelor sursă sau pentru a realiza ajustările necesare la parametri comenzii astfel încât aceasta să poată fi executată cu succes pe un nod server aflat la distanță.

Argumentele din comanda de compilare sunt parcurse secvențial, iar în funcție de conținutul comenzii, aceasta este clasificată: compilare, link-editare sau alt tip. Se extrage executabilul compilatorului folosit pentru a determina dacă face parte din lista celor acceptate, iar toate path-urile relative întâlnite sunt convertite la path-uri absolute.

De asemenea, sunt reținute directoarele specificate pentru localizarea de fișiere header, definite folosind comenzi precum: **-I**, **-iystem** sau **-iquote**. Aceste directoare sunt folosite de către modulul de parsare al dependențelor pentru fișierele sursă: de fiecare dată când este găsită o directivă **#include** în antetul fișierului sursă, numele header-ului inclus este căutat în directoarele specificate folosind comenzile respective.

Extragerea dependențelor pentru fișierele sursă

Modulul implicat în extragerea listei de fișiere header de care depinde compilarea fișierelor sursă pe nodul server construiește un graf complet de dependențe între fișierele proiectului compilat, indiferent de adâncimea ierarhiei de directive **#include**.

Modalitatea de procesare a unui fișier sursă se bazează pe parcurgerea secvențială a conținutului acestuia, pentru a verifica care dintre liniile fișierului conțin directive **#include** sau **#include_next**. După extragerea conținutului acestor tipuri de directive, acestea sunt categorizate:

- cele cu delimitare cu paranteze unghiulare (**<...>**) sunt specifice fișierelor header ce fac parte din librăriile de sistem
- cele cu delimitare pe bază de ghilimele (**"..."**) sunt de obicei folosite pentru includerea de fișiere header locale proiectului

Pe baza categorizării directivelor de includere a fișierelor header, se alterează locațiile la care sunt căutate fișierele respective sau ordinea în care se efectuează căutarea.

După ce fiecare fișier header din codul sursă destinat compilării este extras și stocat, se încearcă găsirea directorului părinte pentru fiecare dependență. În funcție de natura directivei de includere în care a fost găsit un anumit fișier, acesta este căutat prima oară relativ la directorul în care se află fișierul sursă de compilat, iar apoi se verifică locațiile specificate în lista de argumente a apelului către compilator:

- **-iquote** - directoare cu headere locale care sunt incluse cu ghilimele
- **-I** - directoare cu fișiere header ce nu aparțin librăriilor de sistem
- **-isystem** - directoare cu fișiere header din librăriile de sistem

Operațiunile explicate anterior sunt repetate folosind o abordare recursivă, întrucât după ce locația unui fișier header inclus este găsită, fișierul respectiv este parcurs și devine astfel punctul de pornire pentru soluționarea unei noi liste de directive de includere. Această abordare duce la construirea grafului de dependențe folosind o metodă de parcurgere în adâncime (**depth-first**), ceea ce duce și la detectarea dependențelor tranzitive ale fișierului sursă inițial.

Odată ce un fișier este parcurs în întregime, iar toate directivele de includere pe care acesta le cuprinde sunt soluționate, toate informațiile sunt memorate într-un spațiu de stocare cache, astfel încât să nu mai fie nevoie ca fișierul să parsat din nou, la rulări ulterioare, ceea ce are rolul de a îmbunătăți timpul de execuție. Fiecare fișier procesat este caracterizat folosind valoarea hash-ului **SHA1**[35], calculată pe baza conținutului, precum și a dimensiunii totale.

Astfel, dacă un fișier deja a fost parcurs, iar dimensiunea sa și valoarea hash-ului nu s-a modificat de la ultima rulare, informațiile despre fișierul respectiv sunt extrase direct din cache.

Gestiunea comunicației cu nodurile de compilare

Aplicația client joacă rolul de planificator al cererilor de compilare interceptate pe nodurile de compilare detectate în rețea.

Pentru fiecare comandă de compilare care a fost interceptată, aplicația creează o sesiune de compilare cu unul dintre serverele disponibile, care se finalizează cu primirea fișierului obiect pentru fișierul compilat, în caz de succes. Tratarea fiecărei cereri de compilare folosind o sesiune dedicată încurajează crearea unui număr foarte mare de sesiuni independente, care rulează în mod concurent. Adoptarea acestei abordări facilitează paralelizarea procesului de compilare al proiectului, iar numărul maxim de sesiuni de compilare disponibile la un moment de timp este dat de numărul total de sesiuni ce pot fi soluționate de către toate nodurile server care fac parte din sistem.

Înainte de inițierea unei sesiuni de compilare cu unul dintre serverele disponibile, aplicația client parsează comanda de compilare pentru a extrage informațiile relevante,

iar apoi determină lista completă de dependențe pentru compilarea fișierului sursă curent. Comunicarea efectivă începe prin selectarea unui server de compilare cu resurse disponibile, iar lista cu servere disponibile este actualizată în funcție de primirea unor pachete periodice de **"advertising"** de la fiecare dintre nodurile active.

După selectarea unui server valabil, aplicația client inițiază o conexiune TCP cu nodul server ales și transmite o solicitare de creare a sesiunii de compilare, iar datele transmise împreună cu cererea de creare reprezintă informații utilizate pentru a produce un mediu de compilare asemănător cu mediul de rulare de pe nodul client:

- Identificator unic UUIDv4 al nodului client
- Argumentele din linia de comandă pentru apelul de compilare
- Path-ul către directorul din care s-a comandat compilarea
- Lista cu directoare în care sunt căutate fișierele header
- Lista de dependențe pentru compilarea fișierului, alături de metadatele folosite pentru determinarea integrității fișierelor: dimensiunea fișierului, hash-ul SHA1 al conținutului

În momentul primirii cererii de inițiere a sesiunii, serverul verifică validitatea datelor și creează o înregistrare asociată unei noi sesiuni de compilare, asociată cu un identificator unic UUIDv4. Nodul server verifică mai apoi care fișiere din lista de dependențe primită trebuie trimise de către client, întrucât anumite dependențe se pot regăsi deja în stocarea nodului. Transmiterea exclusivă a fișierelor absente reduce semnificativ timpul necesar transferului de dependențe.

După confirmarea sesiunii, clientul trimite toate fișierele solicitate folosind blocuri de dimensiune fixă care încapsulează următoarele informații:

- Identificatorul UUIDv4 de sesiune
- Numărul de secvență pentru blocul respectiv
- Offset-ul blocului în conținutul fișierului trimis
- Marcaj care indică dacă blocul respectiv este ultimul pentru fișierul transferat

După ce nodul server confirmă primirea corectă a unui fișier, clientul continuă în mod automat cu trimiterea blocurilor pentru următorul fișier header. Procesul este repetat până când toate dependențele necesare pentru compilarea fișierului sursă al sesiunii au fost primite de către server.

După finalizarea compilării fișierului sursă pe nodul server, fișierul obiect creat este trimis către nodul client folosind aceeași metodă de transfer cu blocuri, care este utilizată și pentru transmiterea fișierelor dependență. La recepționarea rezultatului, informațiile legate de compilare sunt salvate și trimise către instanța executabilului pentru wrapper-ul de compiler care așteaptă un răspuns ce conține:

- codul numeric de terminare (0 reprezintă succesul, iar orice altă valoare reprezintă eșec)
- Conținutul fluxului **stdout**
- Conținutul fluxului **stderr**
- Durata operațiunii de compilare

Figura 4.10 ilustrează interacțiunea dintre o instanță a wrapper-ului pentru compilator, aplicația client și aplicația server în contextul stabilirii unei sesiuni de compilare pentru un fișier sursă.

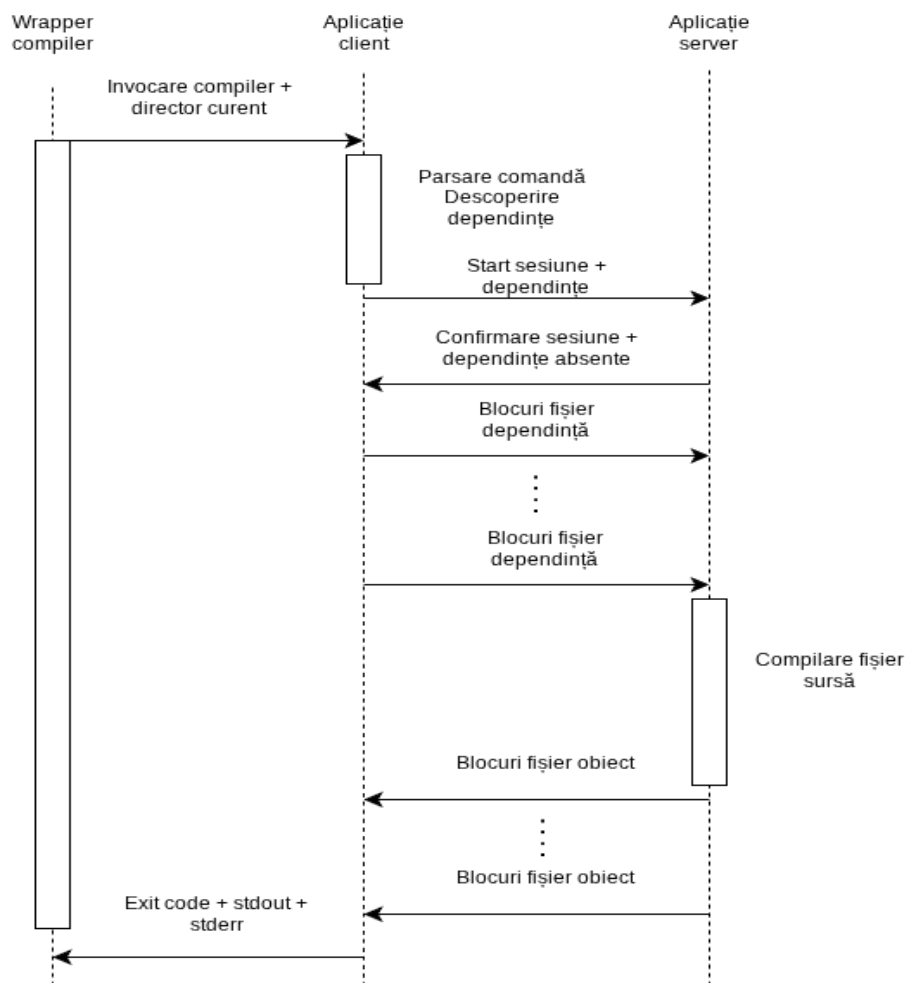


Figura 4.10: Diagrama de secvență pentru interacțiunea dintre wrapper, client și server

4.2.3 Aplicația server

Aplicația server rulează pe nodurile de compilare și are responsabilitatea de a administra resursele de calcul disponibile pe nodul server astfel încât compilarea fișierelor sursă, solicitate de către client, să se desfășoare cu cât mai puțin overhead efectiv. Spre deosebire de aplicația client, a cărei arhitectură a fost prezentată anterior, aplicația server trebuie să gestioneze în mod concurent comunicarea cu unul sau mai mulți clienți, să organizeze pentru fiecare client detectat un mediu de rulare și stocare a fișierelor cât mai apropiat de cel de pe nodul client și să realizeze compilarea fișierelor sursă într-o manieră cât mai eficientă.

Gestiunea nodurilor client și a sesiunilor

De fiecare dată când un nou nod client, necunoscut până acum, încearcă să inițieze o conexiune către serverul de compilare, aplicația server creează o înregistrare care modelează ultima stare cunoscută a clientului. Se centralizează toate informațiile clientului necesare pentru crearea și soluționarea de noi sesiuni de compilare remote:

- Path-ul către directorul de lucru asociat clientului identificat folosind un identificator UUIDv4
- Lista cu toate sesiunile de compilare active al clientului respectiv
- Lista cu starea fișierelor-dependință pentru toate sesiunile de compilare create pentru client

De asemenea, aplicația server ține în evidență și o listă globală cu starea tuturor fișierelor header recepționate de la nodurile client, astfel încât dacă o sesiune de compilare declanșată de un client are nevoie de dependențe care au fost transferate pentru a soluționa o sesiune de compilare din trecut, să nu se mai retrimită dependențele deja existente în stocarea nodului server.

Reconstruirea arborelui de fișiere pentru client

Fiecare nod client are un sistem de fișiere dedicat în cadrul ierarhiei de fișiere de pe nodul server, astfel încât fiecare path detectat în comanda de compilare primită de la client, precum și path-ul directorului unde a fost apelată inițial compilarea, să aibă un corespondent direct în spațiul de stocare al nodului server. Întrucât toate dependențele recepționate pentru clientul în cauză sunt stocate în cadrul ierarhiei de directoare asociate clientului, se menține locația relativă a fișierelor header față de fișierul sursă de compilat în momentul rulării comenzii de compilare. Toată structura de fișiere a proiectului compilat va avea o reprezentare identică în sistemul de fișiere al nodului server. Astfel, în momentul în care compilatorul analizează directivele `#include` în etapa de preprocesare, toate fișierele header necesare pot fi găsite fără apariția erorilor de compilare.

Întrucât rădăcina sistemului de fișiere pentru fiecare client o constituie un director cu numele dat de identificatorul unic UUIDv4 al clientului, se păstrează o izolare complete între mediile de lucru pentru clienți diferiți, ceea ce elimină apariția conflictelor între sesiunile de compilare ale unor clienți diferiți, care depind de fișiere cu același path.

În timpul unei sesiuni de compilare, toate path-urile locale primite de la un client anume sunt transformate în path-uri specifice mediului de lucru pentru acel client, în ierarhia de directoare a nodului server. Spre exemplu, acest path:

```
/home/radu/distbuild/DistBuildSys/src/client_session.hpp
```

, va avea următoarea formă pe nodul server, în urma transformării:

```
/71d4c98e-5f3a-4a67-bd1f-92e8c6b0a73d/.../DistBuildSys/src/client_session.hpp
```

Gestiunea stării dependențelor

Pentru fiecare fișier care a fost folosit ca dependență pe server este menținută o structură care indică disponibilitatea acestuia în spațiul de stocare al nodului server. Aceste informații permit identificarea facilă a dependențelor care deja sunt existent la momentul creării unei noi sesiuni, pentru a se evita retrimiteră inutilă de dependențe redundante pentru sarcinile succesive de compilare. Fiecare fișier are stocată valoarea hash SHA1 caracteristică conținutului și dimensiunea conținutului, pentru a detecta situațiile în care o dependență a fost folosită în trecut pe server, dar și-a modificat conținutul de la ultima utilizare.

Apelarea compilatorului

Executarea comenzilor de compilare este efectuată de către un modul care gestionează toate operațiunile de compilare. Acest modul are rolul de a decupla firele de execuție care rulează sarcinile de comunicare cu nodul client, în cadrul sesiunii, și firele de execuție pe care rulează logica necesară pentru realizarea efectivă a proceselor de compilare. Această abordare oferă certitudinea că indiferent câte sesiuni de compilare sunt aflate în desfășurare, compilarea se efectuează într-un mediu separat, fără a fi afectată performanța de compilare în cazul încărcării însemnate a nodului server.

Din punct de vedere arhitectural, componenta implementează un model clasic **producător-consumator**, unde consumatorii reprezintă o colecție de thread-uri ce fac parte dintr-un **thread pool** a cărui dimensiune este fixă și se setează la pornirea aplicației server. Componentele modului sunt următoarele:

- Producători - instanțele active ale sesiunilor de compilare, care generează job-uri de compilare
- Consumatori - thread-urile din thread pool-ul modului, care preiau cererile și lansează câte un proces nou al compilatorului pentru fiecare cerere

- Coada de job-uri - solicitările de compilare sunt stocate într-o structură **FIFO** până când unul dintre thread-urile din pool își termină sarcina curentă și preia un nou job de compilare din queue

Pentru a menține evidența resurselor de compilare disponibile, de fiecare dată când unul dintre thread-uri preia un job de compilare, un contor atomic, care reprezintă numărul de poziții libere pentru compilare, este decrementat, iar la finalul sarcinii, contorul atomic este incrementat.

Figura 4.11 ilustrează modul în care cererile de compilare sunt preluate de către thread-urile managerului de compilare.

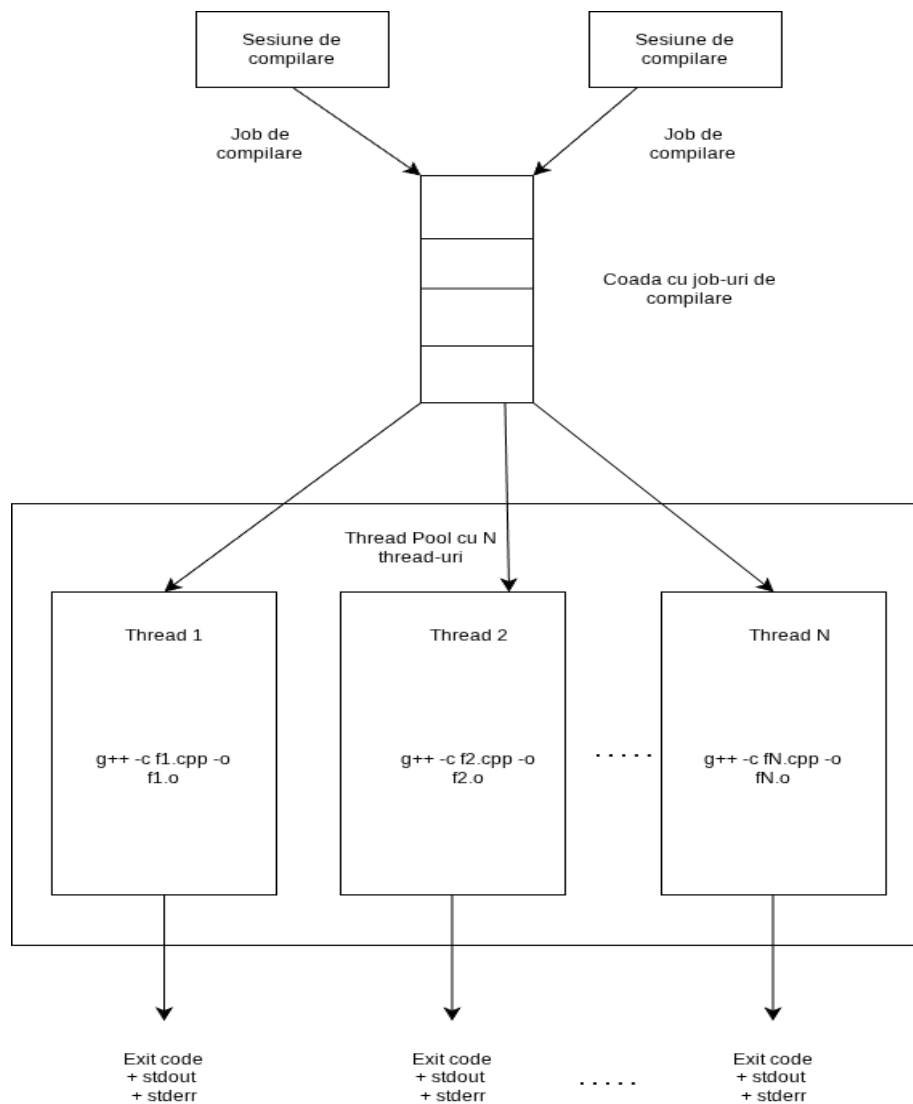


Figura 4.11: Funcționarea managerului de compilare

4.3 Aspecte de implementare

În această secțiune descriem câteva aspecte de relevante care au fost identificate în urma implementării arhitecturii conceptuale, descrisă în secțiunea anterioară. De asemenea, argumentăm motivele din spatele alegerii tehnologiilor utilizate și prezentăm modul în care acestea afectează structura implementării.

4.3.1 Implementarea protocolului de comunicație

Buna funcționare a sistemului de compilare distribuită se bazează pe comunicarea eficientă, în rețea între nodurile client și server participante, ceea ce a creat necesitatea proiectării unui protocol de comunicație cu o structură cât mai expresivă a mesajelor, care să faciliteze implementarea cât mai facilă a unei metode de comunicare performante.

Întrucât s-a optat pentru implementarea componentelor sistemului de compilare distribuit folosind limbajul **C++**, o metodă comodă de a defini maniera de serializare a datelor transferate între client și server o reprezintă utilizarea **Protocol Buffers**[36], dezvoltat de către **Google**.

Protobuf reprezintă un limbaj standardizat de definire a structurii datelor transmise, de obicei în mediul web și reprezintă un înlocuitor pentru arhicunoscutul limbaj **XML**. Protobuf oferă posibilitatea definirii unor structuri de mesaje independente de platforma care le utilizează și stă la baza structurilor de date tranzacționate de framework-uri pentru **remote procedure call** precum **gRPC**[37].

Pentru a defini setul de mesaje folosite pentru implementarea protocolului de comunicare, structura dorită se modelează folosind un limbaj comun, care se adaugă în fișiere cu extensia **.proto**. Un nou tip de mesaj se definește folosind următoarea structură:

Listing 4.1: Definiția unui mesaj în Protobuf

```
message NUME_MESAJ {
    REGULA_MEMBRU TIP_MEMBRU NUME_MEMBRU = NUMAR_ID_MESAJ
    ....
    required string name = 1;
    optional int32 age = 2;
    repeated string email = 3;
}
```

Pentru fiecare membru component al unui tip de mesaje, este obligatoriu să precizăm o regulă de populare, dintre cele trei disponibile: **optional**, **required**, **repeated**. Membrii cu tipul **repeated**, se pot repeta fără o limită superioară, sau pot să nu apară deloc. De asemenea, fiecare membru trebuie să aibă setat un număr de identificare unic, iar valorile cuprinse între 19000 și 19999 sunt rezervate și nu pot să fie folosite.

O altă funcționalitate utilă pusă la dispoziție de limbajul de serializare **Protobuf** este structura **oneof**, ce permite gruparea mai multor tipuri diferite de mesaje într-o structură logică unică. Mecanismul **oneof** devine foarte util în situațiile în care un mesaj poate avea

mai multe structuri posibile, însă doar una singură este valabilă la un anumit moment de timp. Acest mecanism se aseamănă cu keyword-ul **union** din limbajul **C**[38], fiind o unealtă des utilizată în definirea structurii pachetelor pentru protocoalele de comunicație. Exemplul următor prezintă folosirea mecanismului **oneof**, pentru definirea pachetelor de pot fi transmise de către nodurile client ale sistemului distribuit:

Listing 4.2: Mesajele transmise de către nodul client

```
message ClientMessage {
  oneof content {
    ClientSessionStartRequest session_start = 1;
    ClientSessionAbortRequest session_abort = 2;
    ClientFileChunkUploadRequest file_chunk_upload = 3;
  }
}
```

După ce s-a completat un fișier **.proto** cu structura pachetelor care vor fi transferate folosind protocolul de comunicare ales, fișierul **.proto** este transformat într-o serie de fișiere sursă ce conțin instrucțiuni din limbajul de programare folosit pentru dezvoltarea aplicației, în cazul nostru, **C++**. Pentru transformare, se folosește compilatorul **Protoc** (Protocol buffers compiler), care este invocat folosind o astfel de comandă:

```
protoc --cpp_out=./support ./distbuild_messages.proto
```

În urma apelului către compilatorul **protoc**, se creează două fișiere **C++**, un fișier sursă și un fișier header, care conțin clase ce modelează structura descrisă a pachetelor, precum și definiții de metode utilizate pentru a manipula pachetele în cauză. Procesul de compilare pentru fișierele Protobuf este exemplificat de Figura 4.12:

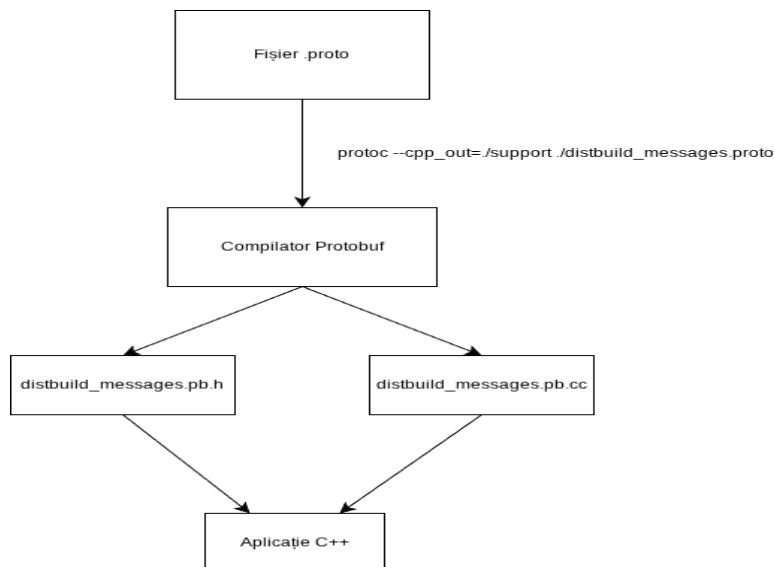


Figura 4.12: Compilarea fișierelor **.proto**

Pe baza arhitecturii conceptuale descrise anterior, tipurile de mesaje transferate în comunicarea dintre nodurile client și cele server sunt următoarele, definite folosind limbajul standardizat **Protobuf**:

Listing 4.3: Mesajele transmise de sistemul de compilare distribuită

```
// Client messages
message ClientMessage {
  oneof content {
    ClientSessionStartRequest session_start = 1;
    ClientSessionAbortRequest session_abort = 2;
    ClientFileChunkUploadRequest file_chunk_upload = 3;
  }
}

message ClientSessionStartRequest {
  string client_id = 1;
  string client_working_dir = 2;
  string client_source_file = 3;
  string client_compiler = 4;
  repeated string client_cmd_line_args = 5;
  repeated string client_idirs = 6;
  repeated FileInfo required_files_info = 7;
}

message ClientSessionAbortRequest {
  string session_id = 1;
  string client_id = 2;
}

message ClientFileChunkUploadRequest {
  string session_id = 1;
  string filename = 2;
  uint64 sequence_no = 3;
  uint64 offset = 4;
  bytes data = 5;
  bool is_last_chunk = 6;
}

// Server messages
message ServerMessage {
  oneof content {
    ServerSessionConfirmedResponse session_confirmed = 1;
  }
}
```

```

        ServerSessionAbortResponse session_abort = 2;
        ServerFileUpCompleteResponse file_up_complete = 3;
        ServerAllReqUploadCompleteResponse all_req_up_complete = 4;
        ServerObjFileChunkResponse obj_file_chunk_transmit = 5;
    }
}

message ServerSessionConfirmedResponse {
    string session_id = 1;
    repeated string required_files = 2;
}

message ServerSessionAbortResponse {
    string session_id = 1;
    string client_id = 2;
}

message ServerFileUpCompleteResponse {
    string session_id = 1;
    string filename = 2;
    bool success = 3;
}

message ServerAllReqUploadCompleteResponse {
    string session_id = 1;
}

message ServerObjFileChunkResponse {
    string session_id = 1;
    int32 compiler_exit_code = 2;
    int32 compiler_call_duration = 3;
    string compiler_stdout = 4;
    string compiler_stderr = 5;
    uint64 filesize = 6;
    bool is_last_chunk = 7;
    bytes data = 8;
}

```

În urma compilării structurilor prezentate cu **Protoc**, rezultă fișierele **distbuild_messages.pb.h** și **distbuild_messages.pb.cc**, care pot fi utilizate în aplicațiile dezvoltate prin includerea fișierului header: **#include "distbuild_messages.pb.h"**.

În ceea ce privește metoda de transport aleasă pentru a susține traficul pachetelor definite anterior între nodurile client și nodurile server, alegerea a fost făcută pe baza unui set minimal de cerințe operaționale. Transferul de fișiere este operațiunea de comunicare care generează volumul cel mai mare de trafic, deci este **absolut necesară** utilizarea unui protocol care:

- să asigure transmiterea cu succes a fiecărui pachet trimis, chiar dacă trebuie ca anumite pachete să fie retransmise din cauza erorilor de transmisie
- să asigure că numeroasele pachete cu blocuri de date din fișierele dependință fragmentate sunt primite tot timpul în aceeași ordine în care au fost trimise, deoarece operațiunea de reordonare a pachetelor primite la nivelul nodului destinatar ar introduce pierderi de performanță la nivelul comunicării
- să fie bazat pe stabilirea unei conexiuni stabile între două entități care comunică, ca să modeleze fidel relația **client-server**, dintre sistemul de calcul care solicită compilarea unui proiect și nodurile care efectuează compilarea efectivă

Având în vedere setul de cerințe enunțat, protocolul de transport ales este **Transmission Control Protocol (TCP)**[39]. TCP pune la dispoziție un mecanism fiabil de transmitere a datelor, întrucât garantează livrarea tuturor pachetelor prin introducerea confirmărilor de primirea și a retransmiterii automate a datelor corupte sau pierdute. Mai mult decât atât, TCP asigură livrarea datelor în aceeași ordine în care au fost trimise, eliminând astfel necesitatea de a implementa mecanisme auxiliare de reordonare a datelor la nivelul aplicației.

Orientarea pe stabilirea unei conexiuni între părțile comunicante permite stabilirea unei sesiuni de comunicație între nodurile client și cele server, înainte de a se începe transferul efectiv de mesaje, fapt ce modelează în mod natural interacțiunea dintre un nod care solicită compilarea și nodurile care execută sarcina de compilare.

În consecință, utilizarea protocolului **TCP** ca modalitate de transport simplifică implementarea protocolului sistemului de compilare distribuită, și facilitează concentrarea asupra operațiunilor specifice logicii de distribuire a compilării, fără a fi necesară asigurarea explicită a fiabilității comunicării.

Înainte ca un mesaj să fie transmis prin conexiunea TCP formată între client și server pentru a efectua compilarea unui fișier sursă, acesta este serializat folosind structurile definite de către compilatorul **Protobuf**, de unde rezultă o serie de octeți. Întrucât **TCP** este un protocol **stream-oriented**, el nu păstrează o delimitare clară între mesajele sistemului, ci transmite un flux continuu de octeți. Din acest motiv, fiecare mesaj trebuie să fie introdus într-un **frame**, prin adăugarea unei secțiuni antet de dimensiune fixă, exprimată folosind un întreg fără semn de 32 de biți, ce conține dimensiunea în octeți a mesajului serializat. Header-ul mesajului este convertit apoi în formatul **network byte order**. Imediat după antetul minimal, se introduce conținutul serializat al mesajului Protobuf.

Structura unui pachet al sistemului de compilare distribuită este ilustrată în Figura 4.13:

4 bytes	N bytes
Lungime mesaj = N	Mesaj Protobuf serializat

Figura 4.13: Structura unui mesaj trimis prin TCP

4.3.2 Asigurarea concurenței în procesare și comunicare

În scenariul obișnuit de utilizare al sistemului de compilare distribuită, se presupune că fiecare client trebuie să gestioneze în mod simultan un număr foarte mare de conexiuni TCP, cu nodurile responsabile de compilare, întrucât fiecare sesiune de compilare necesită deschiderea unei noi conexiuni TCP care să permită transferul fișierelor-dependință necesare compilării, precum și a fișierului obiect rezultat.

Având în vedere această caracteristică a sistemului, devine evidentă nevoia de a implementa operațiunile de procesare și comunicare folosind metode care să asigure **scalabilitatea** și deteriorarea minimă a performanței sistemului pe măsură ce numărul de noduri comunicante crește. Prin urmare am optat pentru utilizarea librăriei **Boost ASIO**[40], pentru **I/O asincron**, care face parte din suita de librării **Boost**[41], pentru limbajul **C++**.

De asemenea, în combinație cu mecanismele de I/O asincron oferite de **Boost ASIO**, implementarea utilizează și **corutinele introduse odată cu standardul C++20**[42], care permit exprimarea logicii asincrone, exprimată în mod normal folosind **funcții callback în cascadă**, într-un mod secvențial și intuitiv, care nu blochează firele de execuție ale aplicației. Integrarea funcțiilor corutină cu librăria Boost ASIO, prin intermediul mecanismelor **co_await** și **co_spawn**, simplifică gestionarea operațiunilor de comunicare în rețea.

Așadar, fiecare sesiune de compilare poate fi implementată elegant sub forma unei ierarhii de corutine, capabilă să își suspende execuția în timp ce operațiunile de scriere sau citire nu pot fi executate, în timp de execuția se reia când apelurile respective pot fi finalizate. Această abordare menține avantajele modelului de comunicare asicron și permite gestionarea în mod eficient a unui număr foarte mare the conexiuni client-server concurente.

Tabelul 4.1 prezintă o comparație între abordările descrise anterior pentru implementarea de operațiuni asincrone folosind Boost ASIO: varianta cu funcții callback apelate la finalizarea operațiilor asincrone și varianta care utilizează corutinele din C++20. În cazul folosirii de callback-uri, logica devine fragmentată, iar logica este construită folosind lanțuri de funcții callback imbricate, ceea ce face codul să fie dificil de urmărit și întreținut.

Funcții callback	Corutine
<pre> void read_message() { auto self = shared_from_this(); async_read(socket_, buffer(&inbound_size_, sizeof(inbound_size_)), [this, self](error_code ec, std::size_t) { if (ec) return handle_error(ec); inbound_size_ = ntohl(inbound_size_); inbound_data_.resize(inbound_size_); read_body(); }); } void read_body() { auto self = shared_from_this(); async_read(socket_, buffer(inbound_data_.data(), inbound_data_.size()), [this, self](error_code ec, std::size_t) { if (ec) return handle_error(ec); handle_message(inbound_data_); read_message(); }); } </pre>	<pre> awaitable<void> run() { try { for (;;) { uint32_t net_size = 0; co_await async_read(socket_, buffer(&net_size, sizeof(net_size)), use_awaitable); size_t size = ntohl(net_size); std::string data(size, '\0'); co_await async_read(socket_, buffer(data.data(), data.size()), use_awaitable); handle_message(data); } } catch (...) { handle_error(); } } </pre>

Tabela 4.1: Boost ASIO cu funcții callback vs. corutine din C++20

Varianta cu corutine permite exprimarea logicii în mod secvențial, ceea ce îmbunătățește lizibilitatea codului.

În continuare, vom prezenta modul în care corutinele sunt utilizate pentru a implementa metoda de primire și de tratare a mesajelor din cadrul unei sesiuni de compilare, folostă atât de către nodurile client, cât și de nodurile server.

Pentru context, vom oferi o porțiune de pseudocod asupra căreia vom aduce explicații pentru a facilita o înțelegere mai bună a interacțiunii dintre nodurile client și server.

Listing 4.4: Ciclul de procesare a mesajelor pentru nodul server

```
awaitable handle_server_session()
{
    while (session is active)
    {
        co_await receive_message(message)

        switch (message.type)
        {
            case SESSION_START:
                update_last_active_time()
                break

            case FILE_CHUNK_UPLOAD:
                update_last_active_time()

                chunk = extract_file_chunk(message)

                success = process_file_chunk(chunk)

                if (!success)
                {
                    send_session_abort()
                }
                break

            case SESSION_ABORT:
                terminate_session()
                co_return

            default:
                terminate_session()
                co_return
        }
    }
}
```


În secțiunea de pseudocod prezentată, cele mai importante aspecte legate funcționarea de procesare a mesajelor dintr-o sesiune de compilare, pentru nodul server, sunt următoarele:

- se utilizează instrucțiunea **co_await** pentru a aștepta în mod non-blocant primirea unui mesaj de la clientul angrenat în interacțiune
- în momentul primirii unui mesaj, execuția se reia automat, fără a fi nevoie de o funcție callback explicită care să trateze evenimentul
- tratarea tuturor tipurilor de mesaje generate de client se face unificat, în cadrul blocului **switch**
- în cazul apariției unei erori (primirea unui tip de mesaj necunoscut, spre exemplu), sesiunea este încheiată în mod controlat, iar ciclul de primire al pachetelor este întrerupt
- bucla de primire a mesajelor este păstrată activă pe toată durata interacțiunii dintre client și server

Capitolul 5

Rezultate teoretice și experimentale

Împreună cu partea de prezentare a proiectului, trebuie să reprezinte aproximativ 70% din lucrare.

Aici sunt prezentate metodele teoretice sau practice de validare/verificare a soluțiilor propuse în partea anterioară, scenariile de testare a corectitudinii funcționale, a utilizabilității, performanței etc.

De asemenea, rezultatele testelor experimentale se pretează unor interpretări și comparații cu rezultatele altor metode similare.

Capitolul 6

Concluzii

6.1 Conținut

- un rezumat al contribuțiilor aduse
- a analiză critică a rezultatelor obținute: avantaje, dezavantaje, limitări
- o descriere a posibilelor dezvoltări și îmbunătățiri ulterioare

6.2 Detalii tehnice

6.2.1 Dimensiune

Cca 3–5% din total.

Bibliografie

- [1] J. R. Powell, “The quantum limit to Moore’s law”, *Proceedings of the IEEE*, vol. 96, nr. 8, pp. 1247–1248, 2008.
- [2] D. Kondo, B. Javadi, P. Malecot, F. Cappello și D. P. Anderson, “Cost-benefit analysis of cloud computing versus desktop grids”, în *2009 IEEE International Symposium on Parallel & Distributed Processing*, IEEE, 2009, pp. 1–12.
- [3] B. Stroustrup, *The C++ Programming Language*, 2013.
- [4] R. C. Martin, “Design principles and design patterns”, *Object Mentor*, vol. 1, nr. 34, p. 597, 2000.
- [5] D. Herity, “C++ in embedded systems: Myth and reality”, *Embedded Systems Programming*, vol. 11, nr. 2, pp. 48–71, 1998.
- [6] H. Rotithor, K. Harris și M. Davis, “Measurement and Analysis of C and C++ Performance”, *Digital Technical Journal*, vol. 10, nr. 1, pp. 32–47, 1999.
- [7] E. Burnette, *Hello, Android introducing Google’s mobile development platform: 2nd*. Android, 2009.
- [8] G. Lim, M. Lee, R. J. Lahaye și Y. I. Eom, “Distributed compilation system for high-speed software build processes”, în *2014 International Conference on Big Data and Smart Computing (BIGCOMP)*, IEEE, 2014, pp. 116–120.
- [9] M. Pool, *distcc, a fast free distributed compiler*, 2003.
- [10] W. Tarreau și D. Rodgman, *LZO stream format as understood by Linux’s LZO decompressor*, 2018.
- [11] S. Ashby, G. Eulisse, S. Schmid și L. Tuura, “Parallel compilation of cms software”, 2004.
- [12] R. Matev, “Fast distributed compilation and testing of large C++ projects”, în *EPJ Web of Conferences*, EDP Sciences, vol. 245, 2020, p. 05 001.
- [13] J. Rosdahl, *ccache - A fast compiler cache*, <https://ccache.dev/>, Accessed: 2025-02-09, 2025.
- [14] J. Rosdahl și Contributors, *ccache Manual: How ccache Works*, Accessed: 2025-02-09, 2025.

- [15] Mozilla și Contributors, *sccache - Shared Compilation Cache*, <https://github.com/mozilla/sccache>, Accessed: 2025-02-09, 2025.
- [16] T. Saario, “Enabling Sccache In CI System”, 2020.
- [17] K. Jo, S. W. Kim și J.-K. Kim, “DiSCo: Distributed scalable compilation tool for heavy compilation workload”, *IEICE TRANSACTIONS on Information and Systems*, vol. 96, nr. 3, pp. 589–600, 2013.
- [18] I. Ltd., *Incredibuild - Accelerate Development with Build Distribution*, <https://www.incredibuild.com/>, Accessed: 2025-02-09, 2025.
- [19] M. Mizuno, L. Chen și V. Wallentine, “Synchronization in a Thread-Pool Model and its Application in Parallel Computing.”, în *PDPTA*, Citeseer, 2003, pp. 1879–1885.
- [20] R. Buyya et al., “High performance cluster computing: Architectures and systems (volume 1)”, *Prentice Hall, Upper SaddleRiver, NJ, USA*, vol. 1, nr. 999, p. 29, 1999.
- [21] M. Van Steen, “Distributed systems principles and paradigms”, *Network*, vol. 2, nr. 28, p. 1, 2002.
- [22] J. Reguera-Salgado și J. A. Rufes, *Asynchronous Programming with C++: Build blazing-fast software with multithreading and asynchronous programming for ultimate efficiency*. Packt Publishing Ltd, 2025.
- [23] R. Kızılarlan. “Parallel Programming with Modern Operating Systems I: Introduction Modern OS and Processors”. (2024), adresa: <https://medium.com/@recepzkilarlan/parallel-programming-with-modern-operating-systems-i-introduction-modern-os-and-processors-58582d77567e> (accesat în 25.6.2026).
- [24] H. Nenov și S. Todorov, “Asynchronous non-blocking IO model approach to avoid the problem C10K in web servers for static content”,
- [25] A. Williams, *C++ concurrency in action*. Simon și Schuster, 2019.
- [26] D. Schmidt și S. D. Huston, *C++ Network Programming, Volume I: Mastering Complexity with ACE and Patterns*. FT Press, 2001.
- [27] P. Diehl, S. R. Brandt și H. Kaiser, “Shared memory parallelism in modern C++ and HPX”, în *Workshop on Asynchronous Many-Task Systems and Applications*, Springer, 2023, pp. 27–38.
- [28] B. Belson, W. Xiang, J. Holdsworth și B. Philippa, “C++ 20 coroutines on microcontrollers—what we learned”, *IEEE Embedded Systems Letters*, vol. 13, nr. 1, pp. 9–12, 2020.
- [29] Tutorialspoint, *Compilation Process in C*, <https://www.tutorialspoint.com/cprogramming/c-compilation-process.htm>, Accessed: 2026-06-27, 2025.
- [30] R. M. Stallman, R. McGrath și P. Smith, “GNU make”, *Free Software Foundation, Boston*, 1988.
- [31] R. M. Stallman, R. McGrath și P. D. Smith, *GNU make*. 2001.

- [32] R. Świdziński, *Modern CMake for C++: Discover a better approach to building, testing, and packaging your software*. Packt Publishing Ltd, 2022.
- [33] M. Pool și Contributors, *distcc - Distributed C/C++ compilation*, <https://www.distcc.org/>, Accessed: 2025-02-09, 2025.
- [34] W. R. Stevens și S. A. Rago, *Advanced programming in the UNIX environment*. Addison-Wesley, 2013.
- [35] D. Eastlake 3rd și P. Jones, “US secure hash algorithm 1 (SHA1)”, rap. teh., 2001.
- [36] K. Varda, *Protocol Buffers: Google’s Data Interchange Format*, Google Open Source Blog, Accessed: 2026-06-29, Iul. 2008. adresa: <https://opensource.googleblog.com/2008/07/protocol-buffers-googles-data.html>.
- [37] K. Indrasiri și D. Kuruppu, *gRPC: up and running: building cloud native applications with Go and Java for Docker and Kubernetes*. O’Reilly Media, 2020.
- [38] D. M. Ritchie, S. C. Johnson, M. Lesk, B. Kernighan et al., “The C programming language”, *Bell Sys. Tech. J*, vol. 57, nr. 6, pp. 1991–2019, 1978.
- [39] V. Cerf și R. Kahn, “A protocol for packet network intercommunication”, *IEEE Transactions on communications*, vol. 22, nr. 5, pp. 637–648, 1974.
- [40] C. Kohlhoff, “Boost. asio”, *Online: http://www.boost.org/doc/libs/1*, vol. 48, nr. 0, pp. 2003–2013, 2013.
- [41] Boost C++ Libraries, *Boost C++ Libraries*, <https://www.boost.org/libraries/latest/grid/>, Accessed: 2026-06-29, 2026.
- [42] C. S. Committee et al., “ISO International Standard ISO/IEC 14882: 2020, Programming Language C++”, *Geneva, Switzerland: International Organization for Standardization (ISO).*, *Tech. Rep*, 2020.

Anexa A

Pseudo-cod sau cod (dacă există)

```
/** Maps are easy to use in Scala. */
object Maps {
  val colors = Map("red" -> 0xFF0000,
                   "turquoise" -> 0x00FFFF,
                   "black" -> 0x000000,
                   "orange" -> 0xFF8040,
                   "brown" -> 0x804000)

  def main(args: Array[String]) {
    for (name <- args) println(
      colors.get(name) match {
        case Some(code) =>
          name + " has code: " + code
        case None =>
          "Unknown color: " + name
      }
    )
  }
}
```